System for Artificial Intelligence

Machine Learning Compilation

Siyuan Feng
Shanghai Innovation Institute

OUTLINE

01 ML Compilation Overview

02 Compilation Flow and Optimization

03 AI Compiler Systems

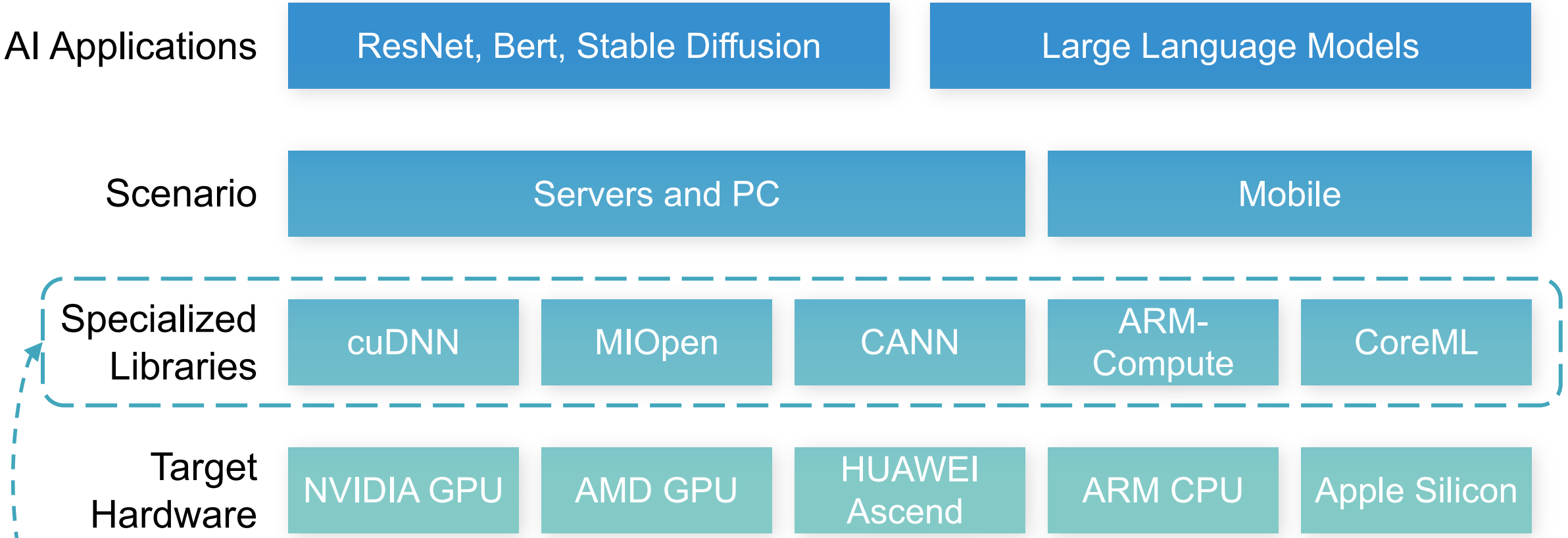
04 Agents and Compiler Co-Design

01

Overview of ML Compilation

Manual Libraries Must Cover Many Targets

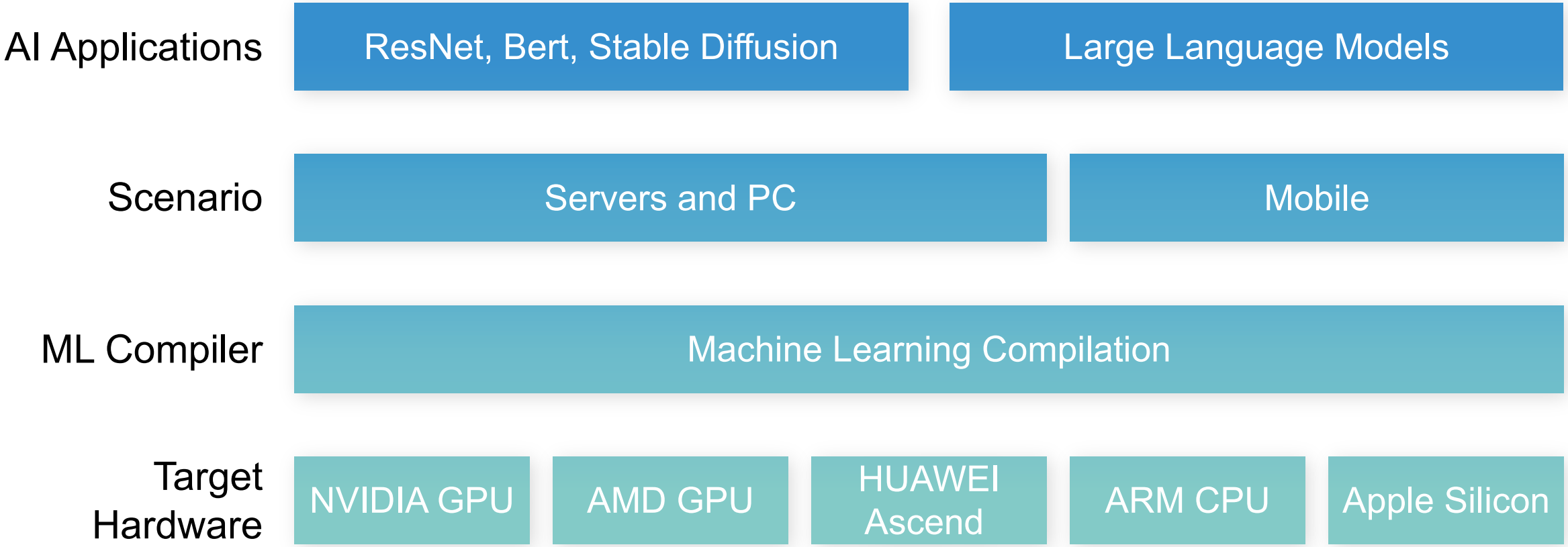
上海创智学院
Shanghai Innovation Institute



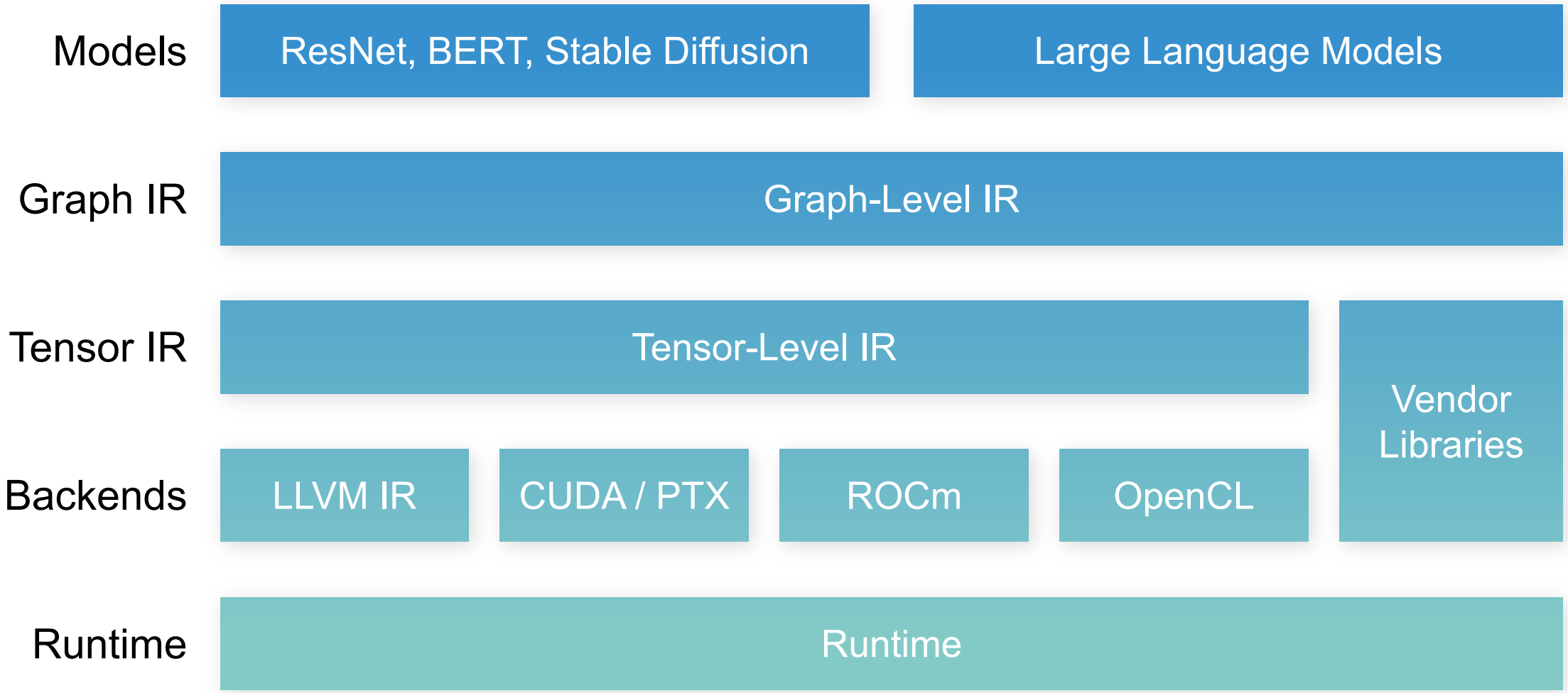
Specialized libraries for each backend (labor intensive)
Hard to be optimal for various models

Compiling Models for Multiple Targets

上海创智学院
Shanghai Innovation Institute



ML Compilation Abstractions



ML Compilation Goals

上海创智学院
Shanghai Innovation Institute

- Equivalent implementations can have different costs. ML compilation must preserve the required semantics while improving execution:
  - Reduce memory traffic and storage.
  - Improve latency or throughput.
  - Control compilation and tuning overhead.
  - Scaling to multiple heterogeneous nodes.

What an IR Makes Explicit

上海创智学院
Shanghai Innovation Institute

Information	Example	Why it matters
Shape and dtype	A: M × K, FP32	Legal operations and storage
Data dependence	ReLU consumes GEMM output	Transformation legality
Storage and effects	Read A; write C	Reuse and safe scheduling
Target constraints	Tile size; supported operation	Feasible code generation

An IR preserves the information needed by its transformations.

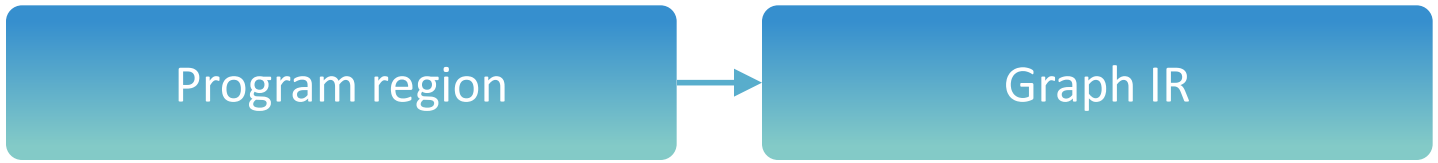
Capturing a Program

上海创智学院
Shanghai Innovation Institute

Import / export

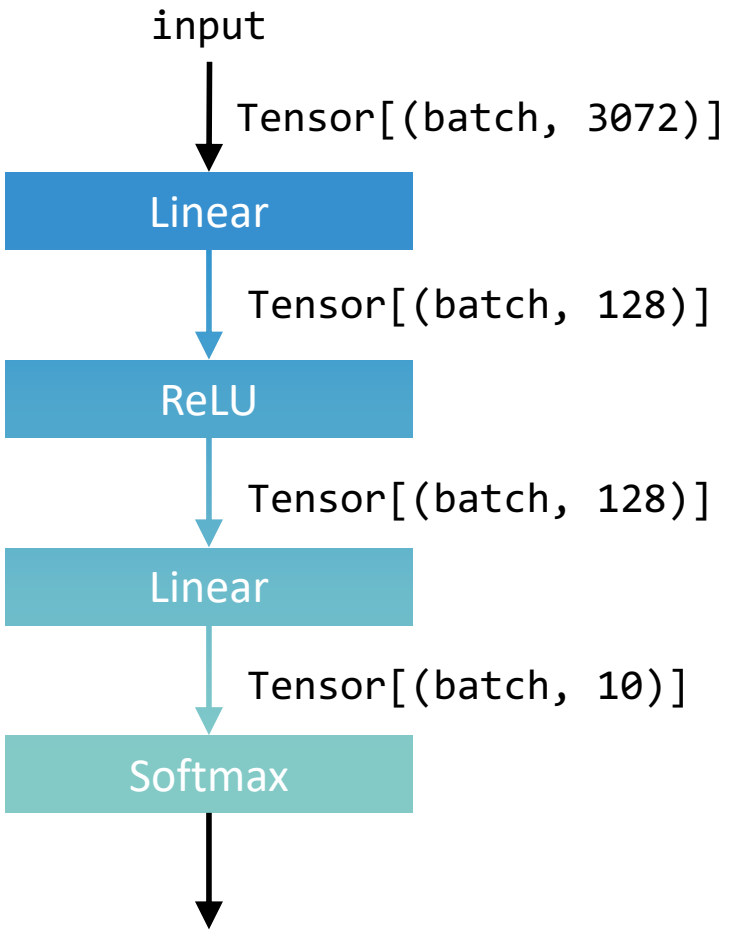
Runtime capture

```
y = x @ w + b
if x.shape[0] == 32:
    y = relu(y)
return y
```



Capture determines which program region is available for optimization.

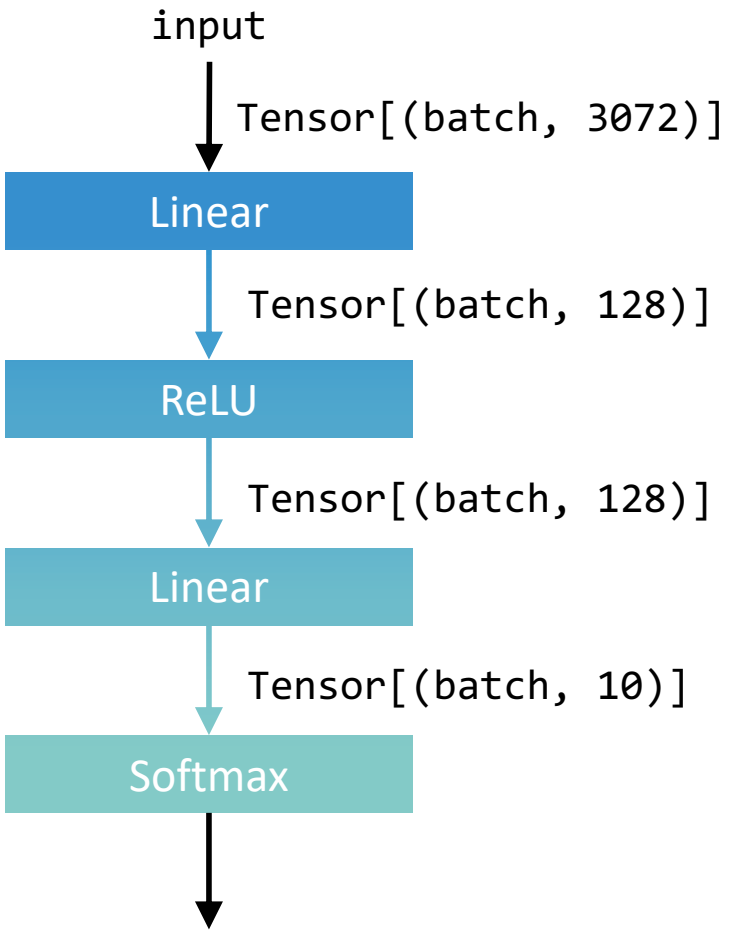
Key Elements in Machine Learning Compilation



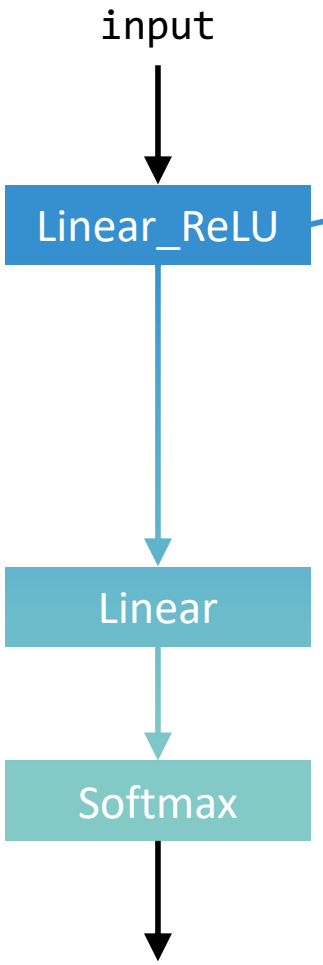
Tensor: multi-dimensional array that stores the input, output and intermediate results of model executions.

Tensor Functions that encodes computations among the input/output. Note that a tensor function can contain multiple operations

Fusing Linear and ReLU



Development



Deployment

```
def linear_relu(x, w, out):
  for i in range(1):
    for j in range(128):
      out[i, j] = 0
      for k in range(3072):
        out[i, j] += x[i, k] * w[j, k]
      out[i, j] = max(out[i, j], 0)
```

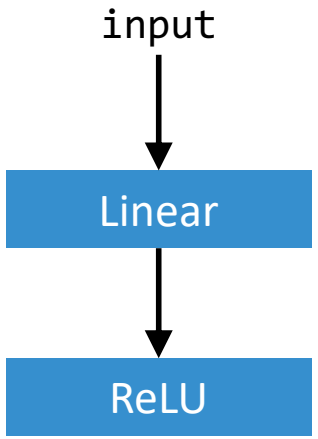
In this particular example, two tensor functions are folded into one (linear-relu). With a specialized implementation (in reality, they will be implemented using low-level primitives).

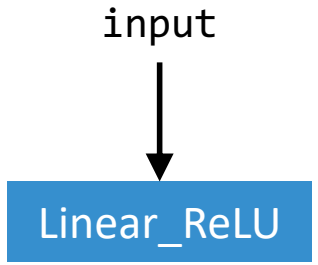
Code example: batch = 1, output width = 128; bias omitted.

One Computation, Multiple Representations

上海创智学院 Shanghai Innovation Institute

- **Abstraction** refers to different ways to represent the same system interface (tensor function)



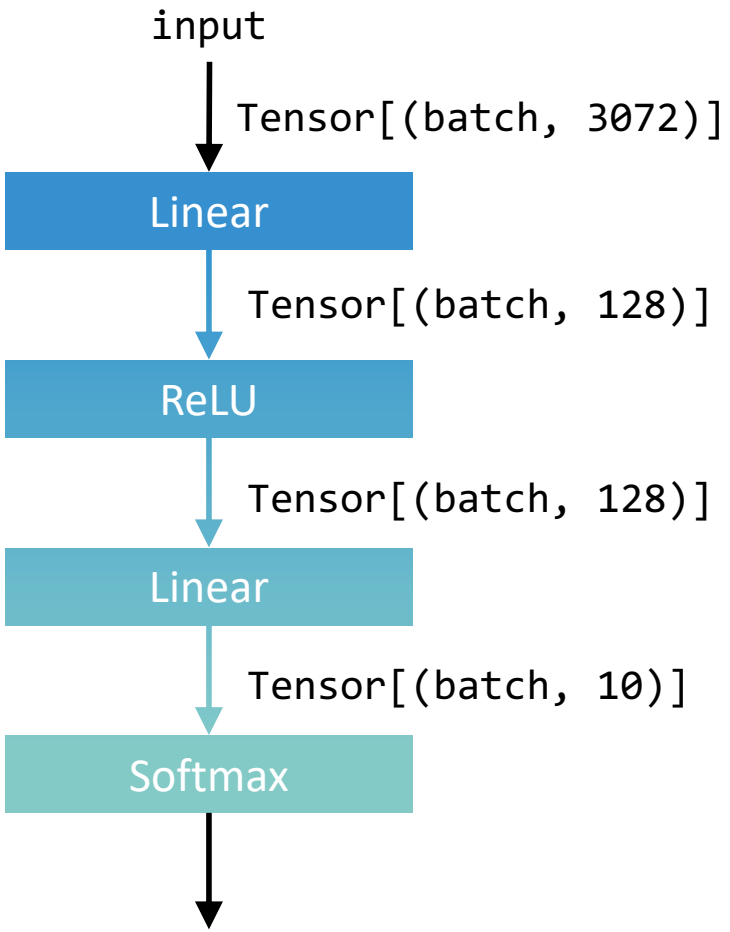



```
def linear_relu(x, w, out):
  for i in range(1):
    for j in range(128):
      out[i, j] = 0
      for k in range(3072):
        out[i, j] += x[i, k] * w[j, k]
      out[i, j] = max(out[i, j], 0)
```

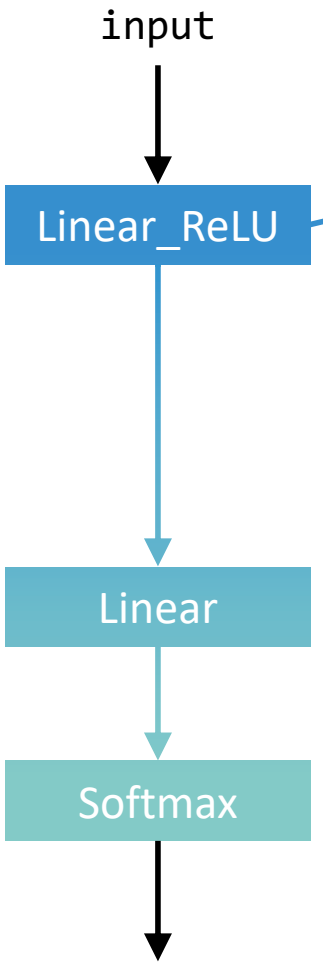
- Three abstraction ways to represent the same tensor function (`linear_relu`), each providing a different level of details. In practice, we usually say that the more specialized version is an **implementation** of higher-level abstraction.

Code example: batch = 1, output width = 128; bias omitted.

Compilation Transforms Tensor Functions



Development



Deployment

```
def linear_relu(x, w, out):
  for i in range(1):
    for j in range(128):
      out[i, j] = 0
      for k in range(3072):
        out[i, j] += x[i, k] * w[j, k]
      out[i, j] = max(out[i, j], 0)
```

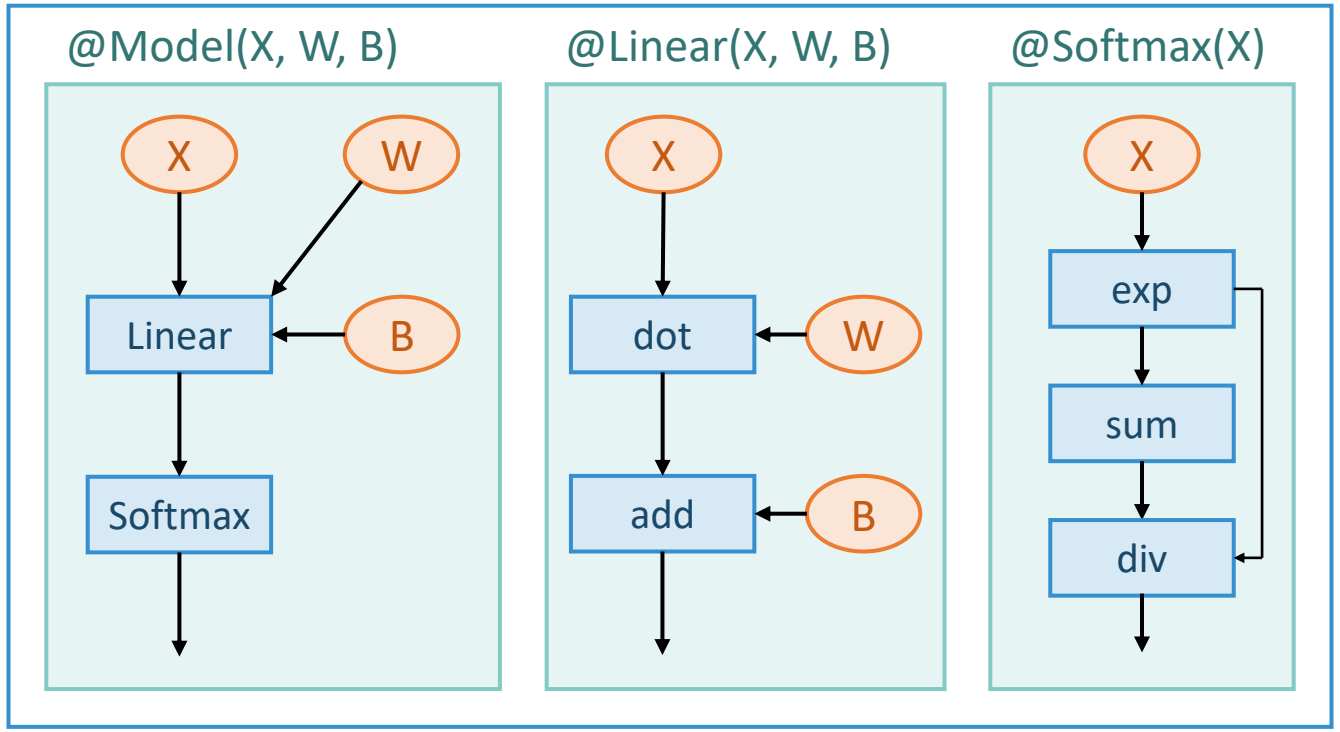
Most MLC process can be viewed as transformation among tensor functions (that can be represented with different abstractions).

Code example: batch = 1, output width = 128; bias omitted.

02

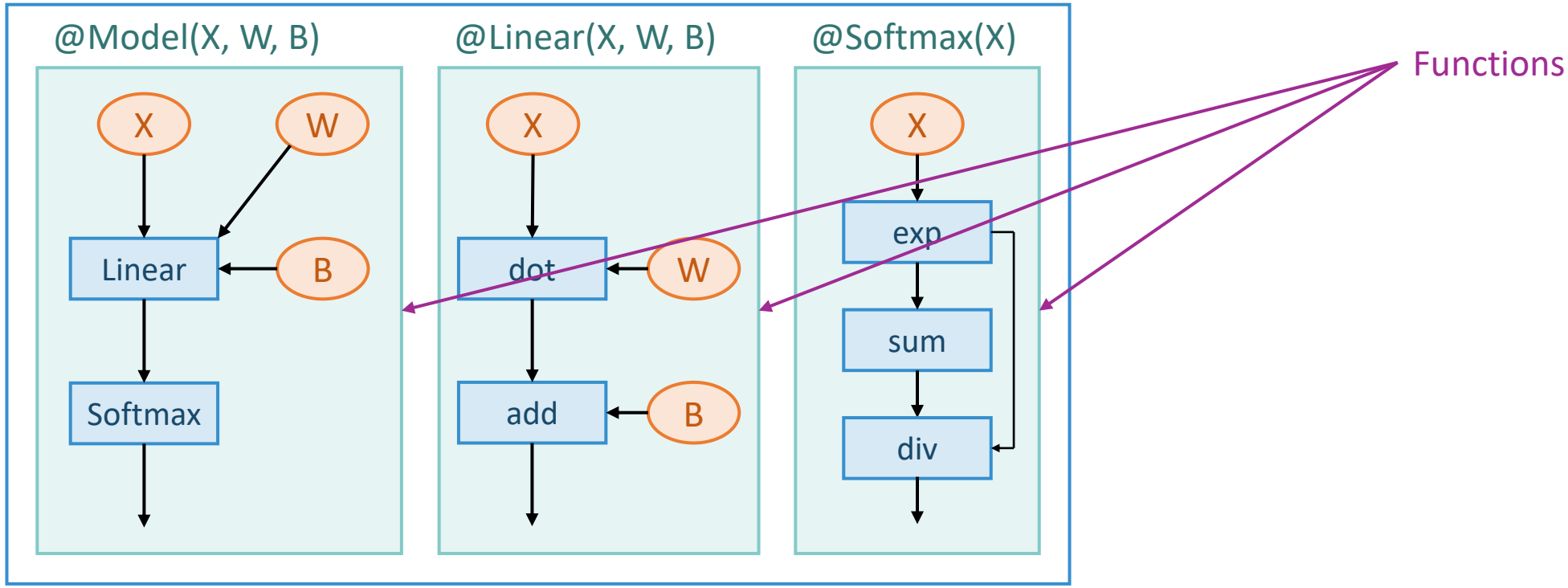
Example ML Compilation Flow

IRModule Organizes Model Functions

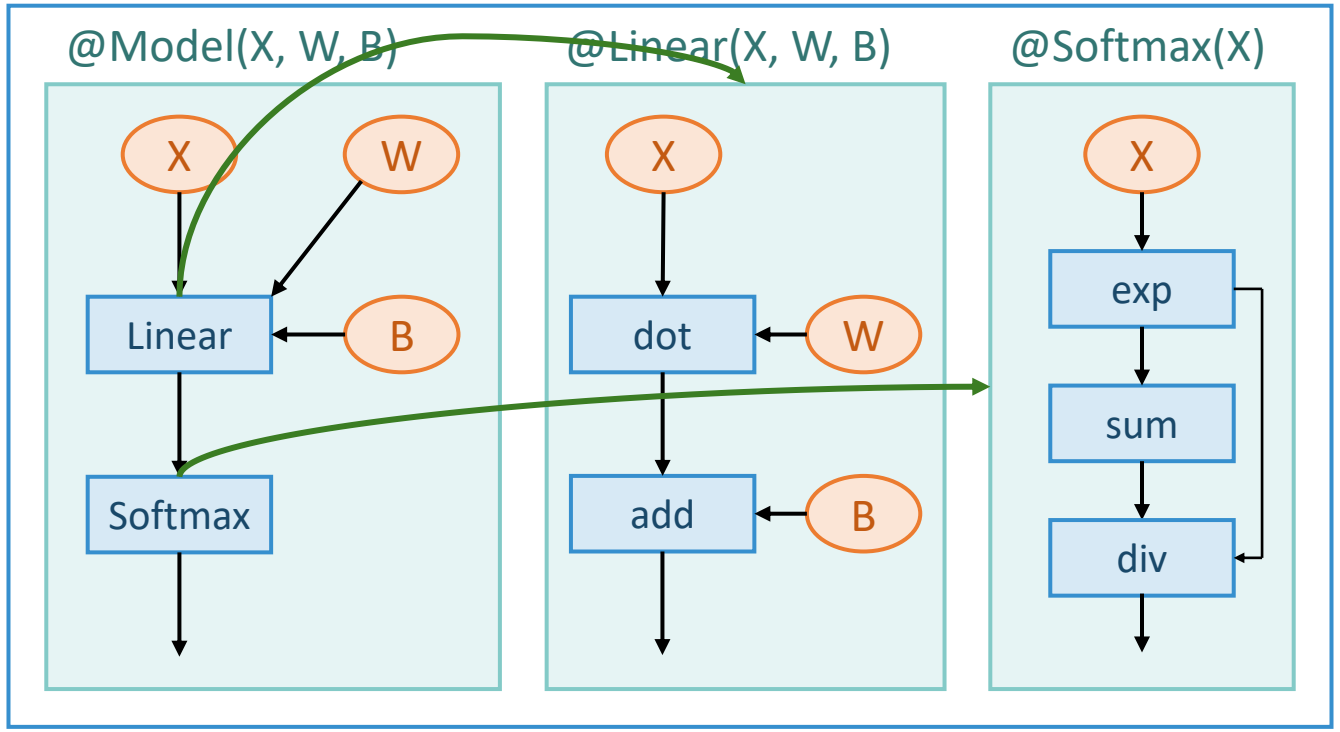


IRModule:
a collection of interdependent functions

Functions Encapsulate Computations



Function Calls Connect Computations

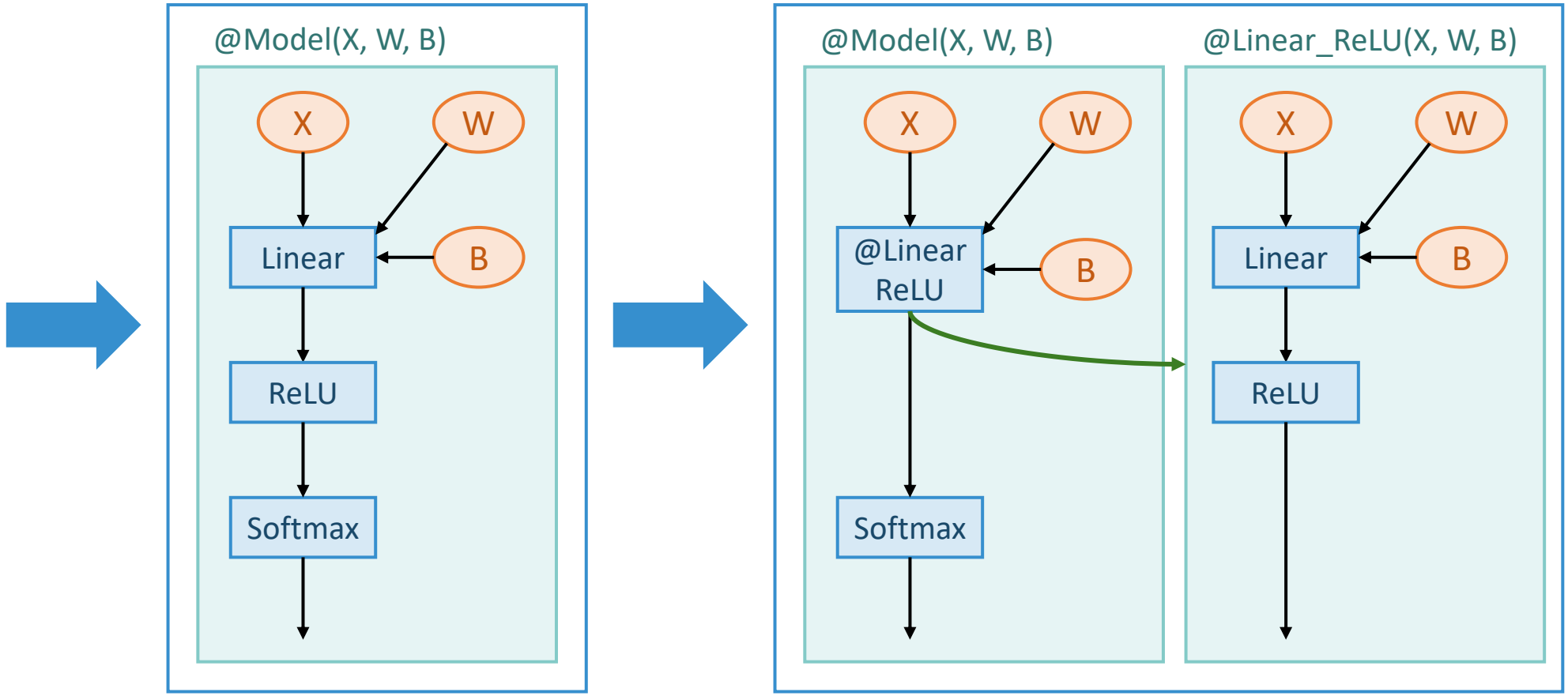


Function calls between models and functions

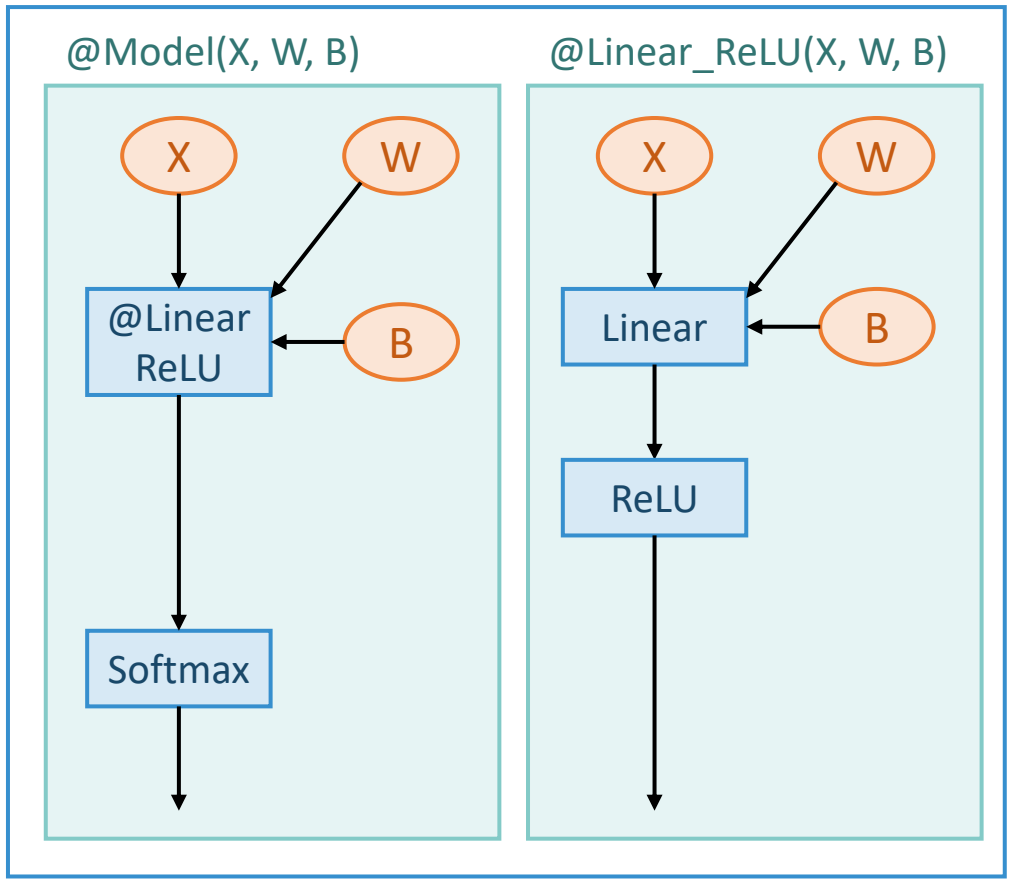
Graph Fusion Combines Linear and ReLU

上海创智学院
Shanghai Innovation Institute

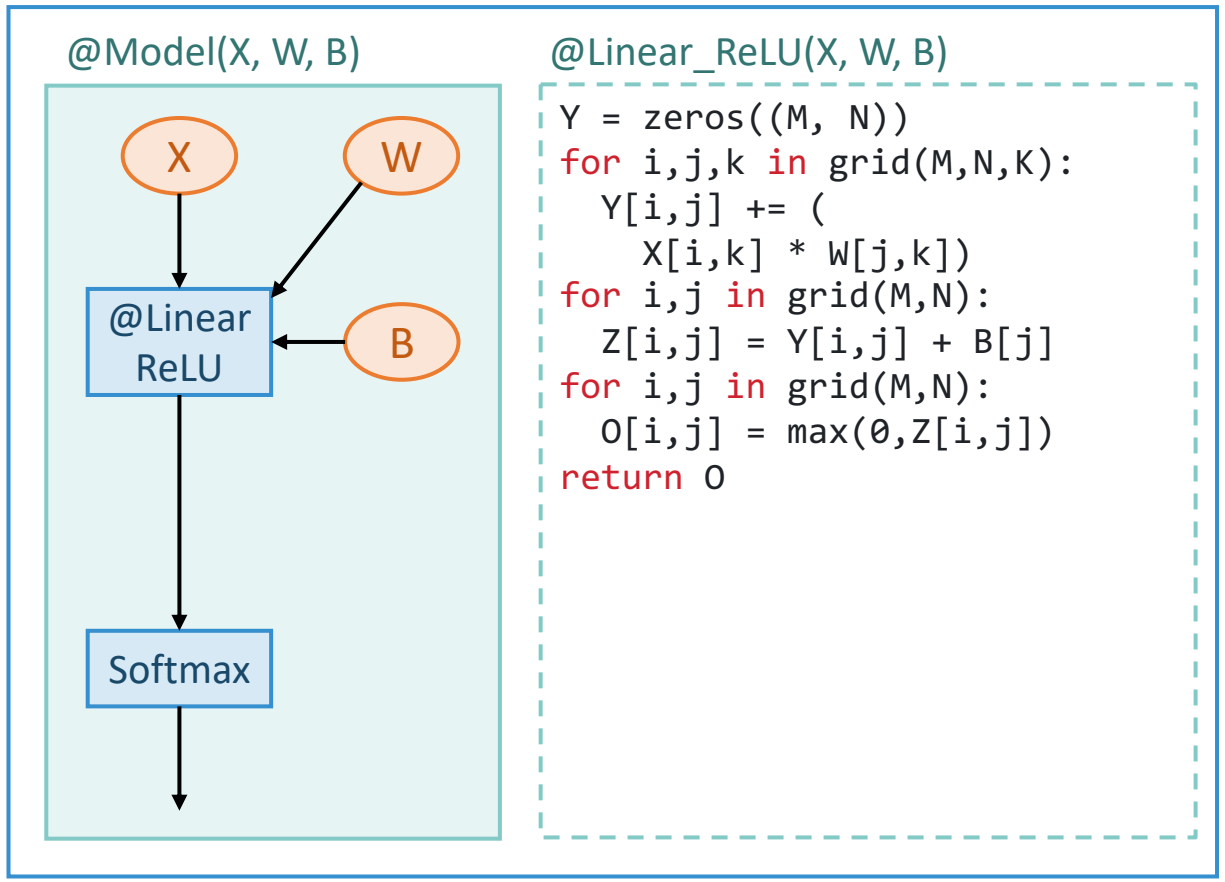
Models



Lowering Exposes Loops and Buffers

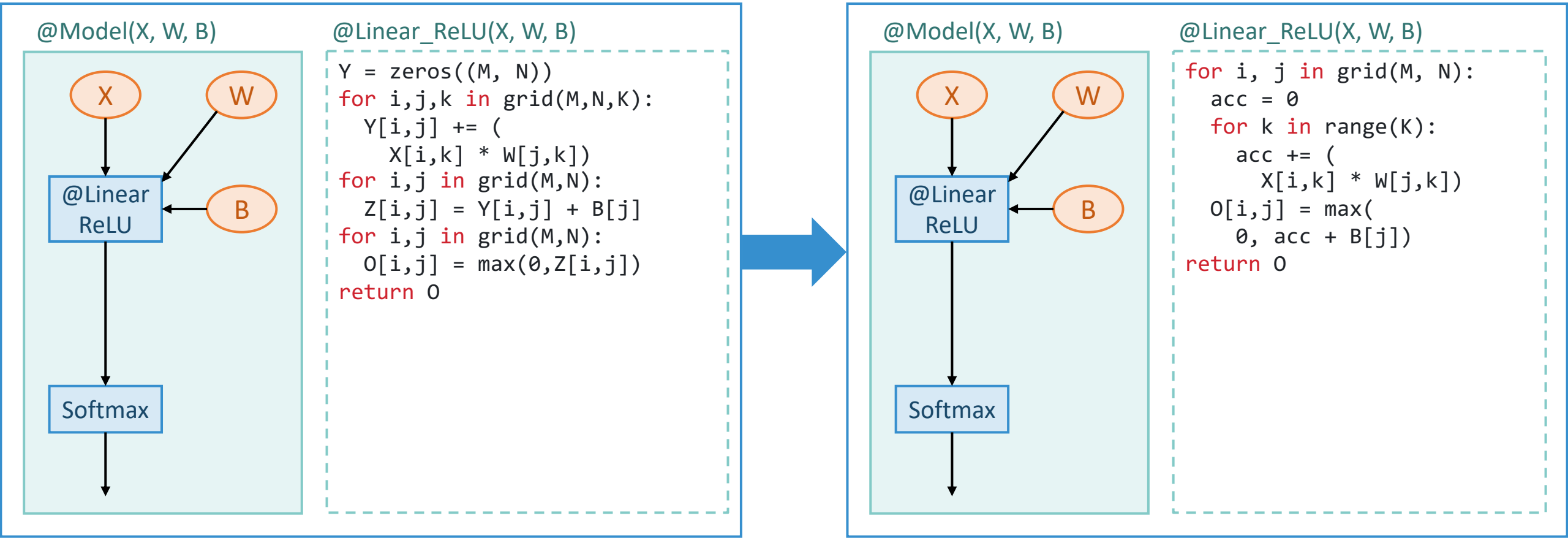


IRModule



IRModule

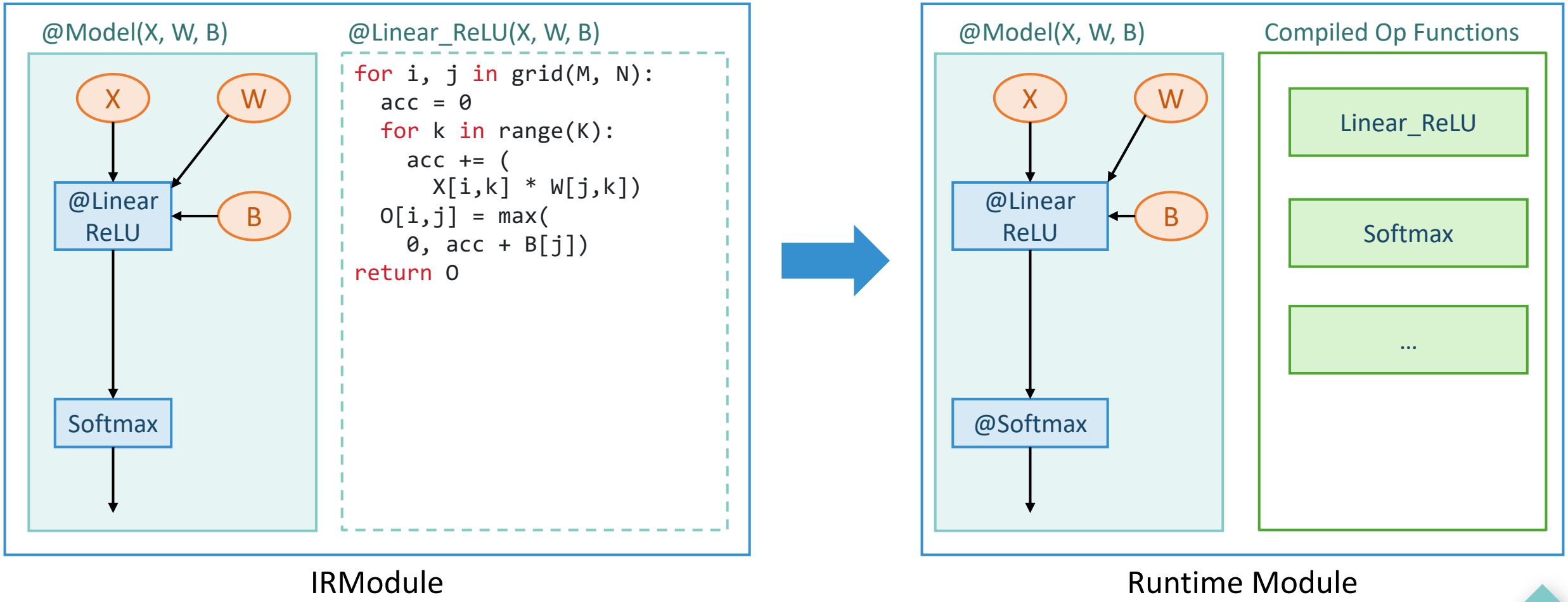
Loop Fusion Eliminates Intermediates



IRModule

IRModule

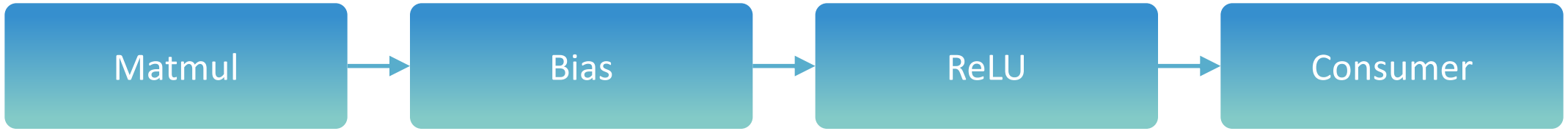
Code Generation Builds Executable Functions



IRModule

Runtime Module

Graph-Level Optimization



Operator patterns

Layout decisions

Storage lifetimes

Graph structure exposes opportunities across operators.

Fusion and Memory Traffic

上海创智学院
Shanghai Innovation Institute

Three kernels: HBM read + write



24N bytes

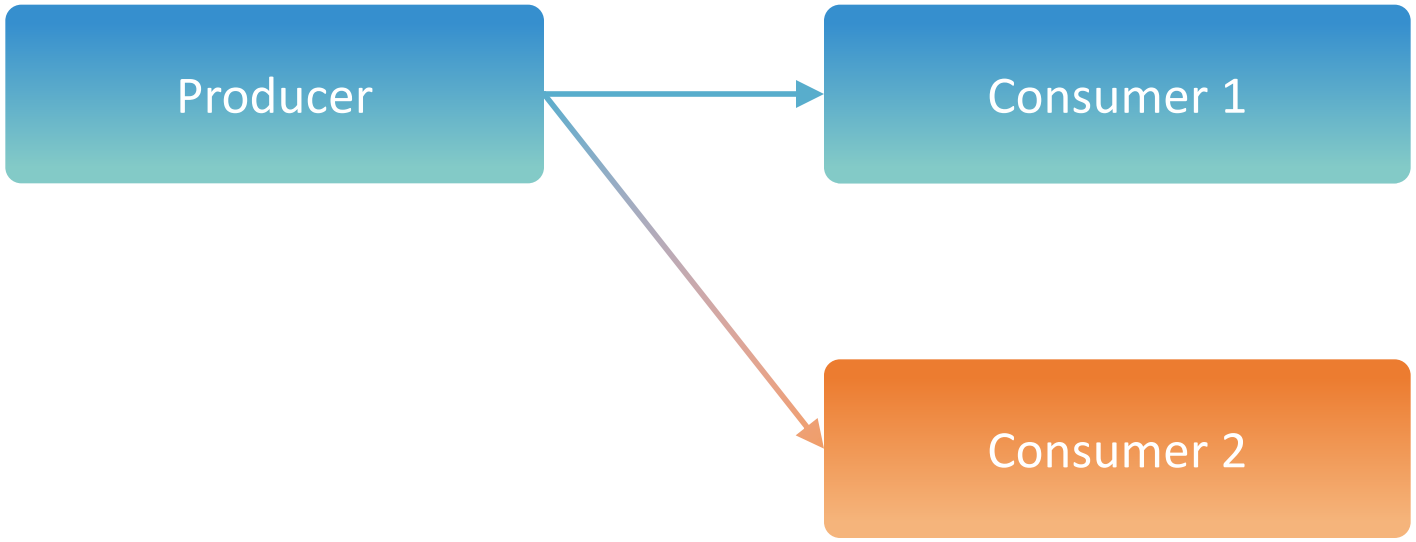
One fused kernel



8N bytes

FP32 teaching model. Scalar a,b resident; no cross-kernel cache hits.

Fusion: Legality and Profitability



Still needed

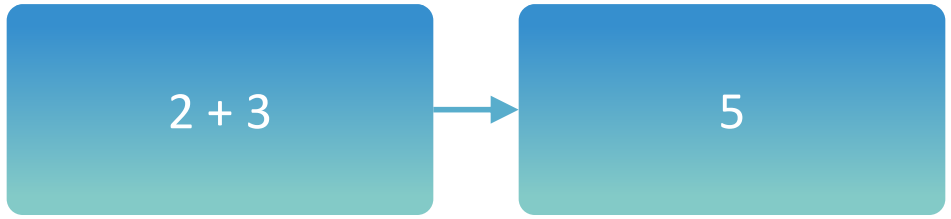
More live data can raise register pressure.

A legal fusion still needs a performance argument.

Graph Rewriting

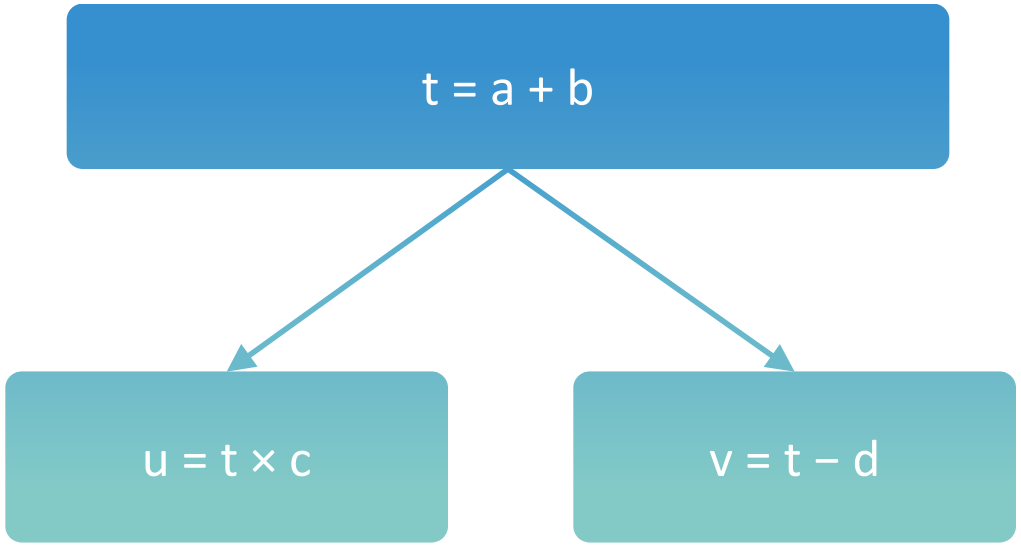
上海创智学院
Shanghai Innovation Institute

Constant folding



Pure integer operations

Common subexpression



Rewrite rules carry conditions, not just matching shapes.

Layout Choices Have a Cost

上海创智学院
Shanghai Innovation Institute

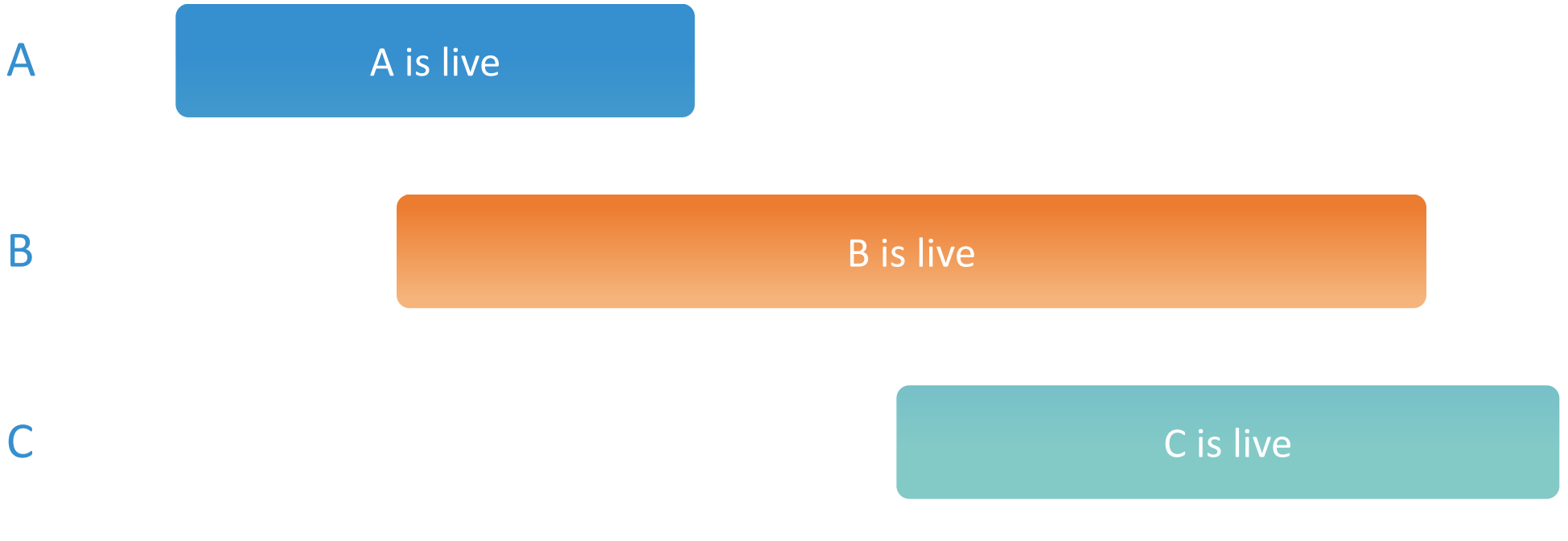
	Original layout	Convert once + reuse
One call	5 ms	3 + 3 = 6 ms
Four calls	4 × 5 = 20 ms	3 + 4 × 3 = 15 ms

Teaching timings. Converted data remains unchanged across calls.

Count conversion, reuse and kernel execution together.

Planning Intermediate Storage

A and C may share a buffer

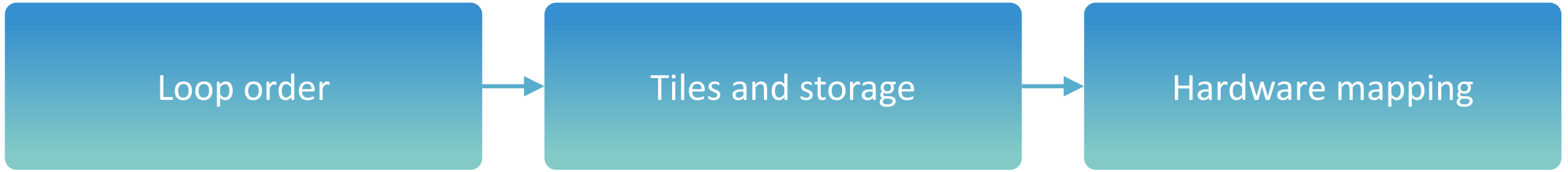


Reuse requires non-overlapping lifetimes and compatible storage.

Computation and Schedule

上海创智学院
Shanghai Innovation Institute

$$C_{ij} = \sum_{k=0}^{K-1} A_{ik} B_{kj}$$



The computation defines values; the schedule organizes execution.

Loop Splitting and Reordering

上海创智学院
Shanghai Innovation Institute

```
for i in range(16)
```

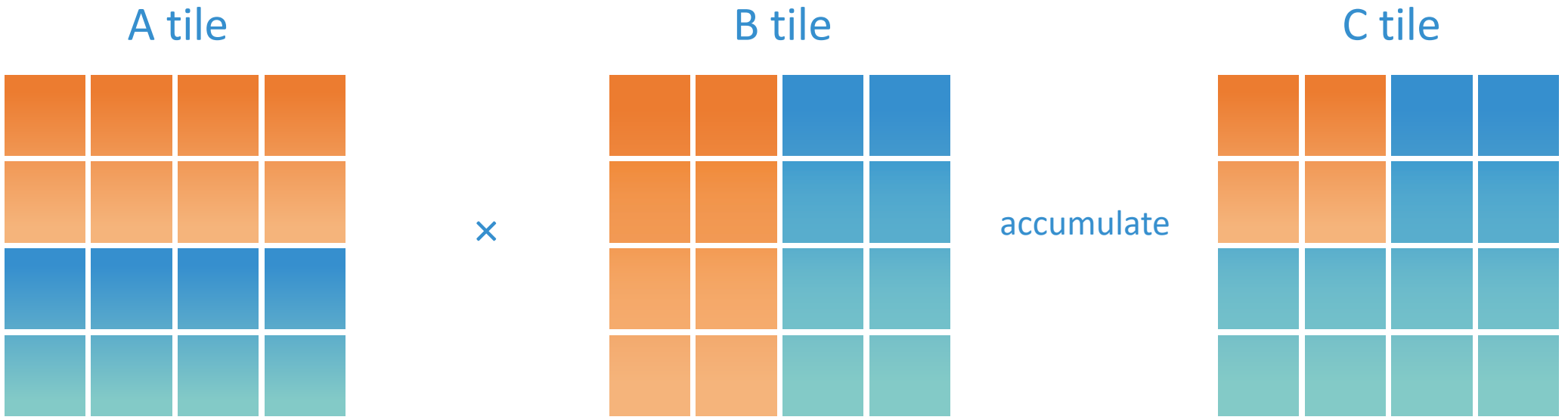
$i = 4 \times io + ii$

0	1	2	3
4	5	6	7
8	9	10	11
12	13	14	15

```
for io in range(4):
    for ii in range(4):
        i = 4*io + ii
```

A valid mapping preserves coverage and required dependences.

Tiling and Data Reuse



Reuse inputs across several output elements

A tile makes reuse explicit and consumes finite local storage.

Mapping Work to Hardware

上海创智学院
Shanghai Innovation Institute

Independent output tiles

T0	T1	T2	T3
T4	T5	T6	T7

Target-specific mapping

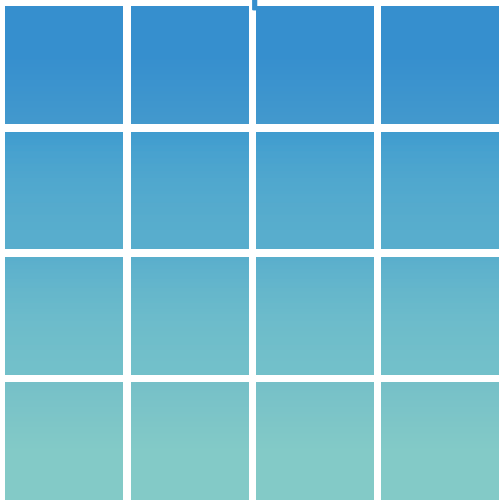
Program instances → Block / lane work

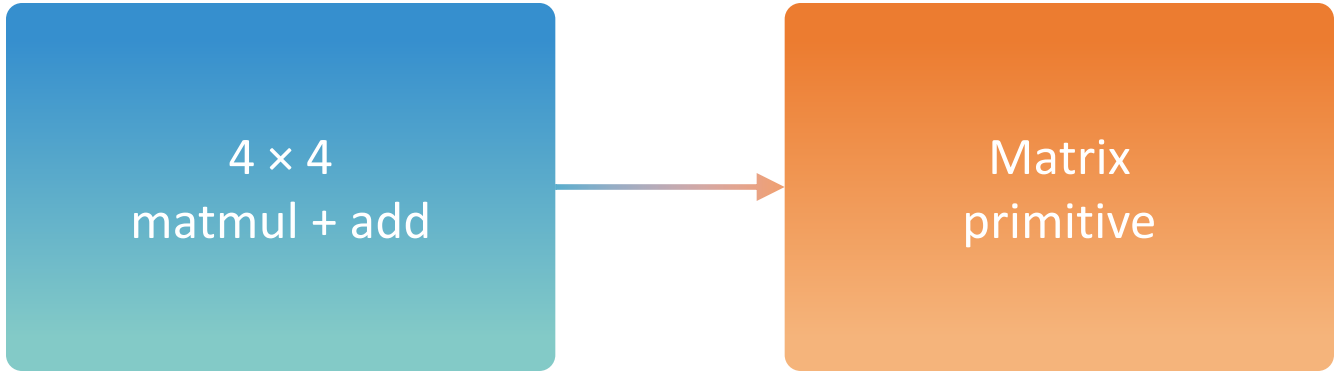
Parallel axes, memory access and resource limits must agree.

Tensorization Maps Tiles to Instructions

上海创智学院
Shanghai Innovation Institute

Matched subcomputation





Shape, dtype, layout and accumulation must match.

Illustrative 4 × 4 primitive; actual instruction contracts vary.

Discussion: Schedule Validity

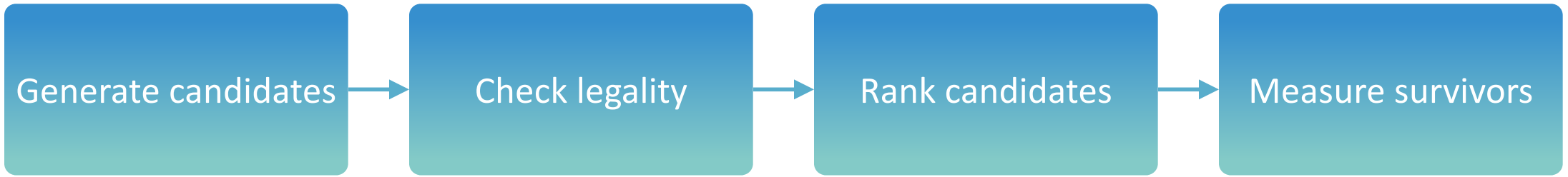
上海创智学院
Shanghai Innovation Institute

A 10 × 10 output uses 4 × 4 tiles.

1. How many output tiles are needed?
2. Which tiles need boundary handling?
3. Can reduction steps run independently?

Discuss output coverage, bounds and reduction dependences.

Automatic Schedule Search



Search quality depends on its space, feedback and budget.

Cost Models and Measurements

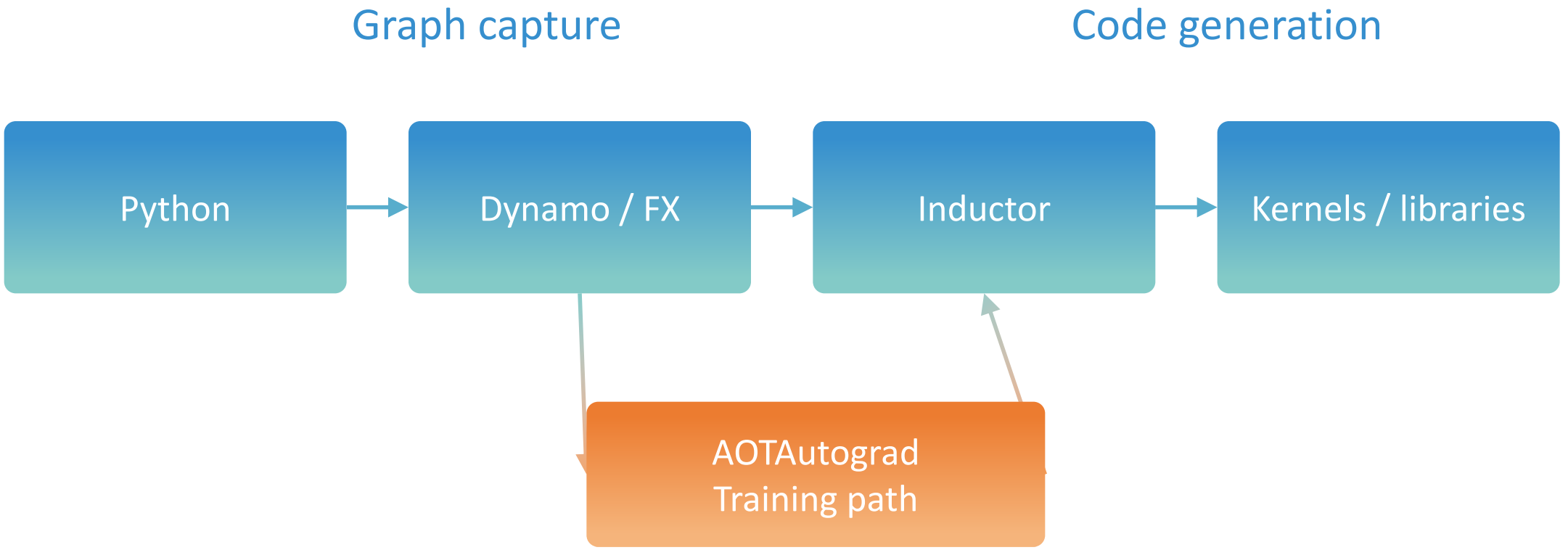


Illustrative ranking

Estimates can miss hardware effects.

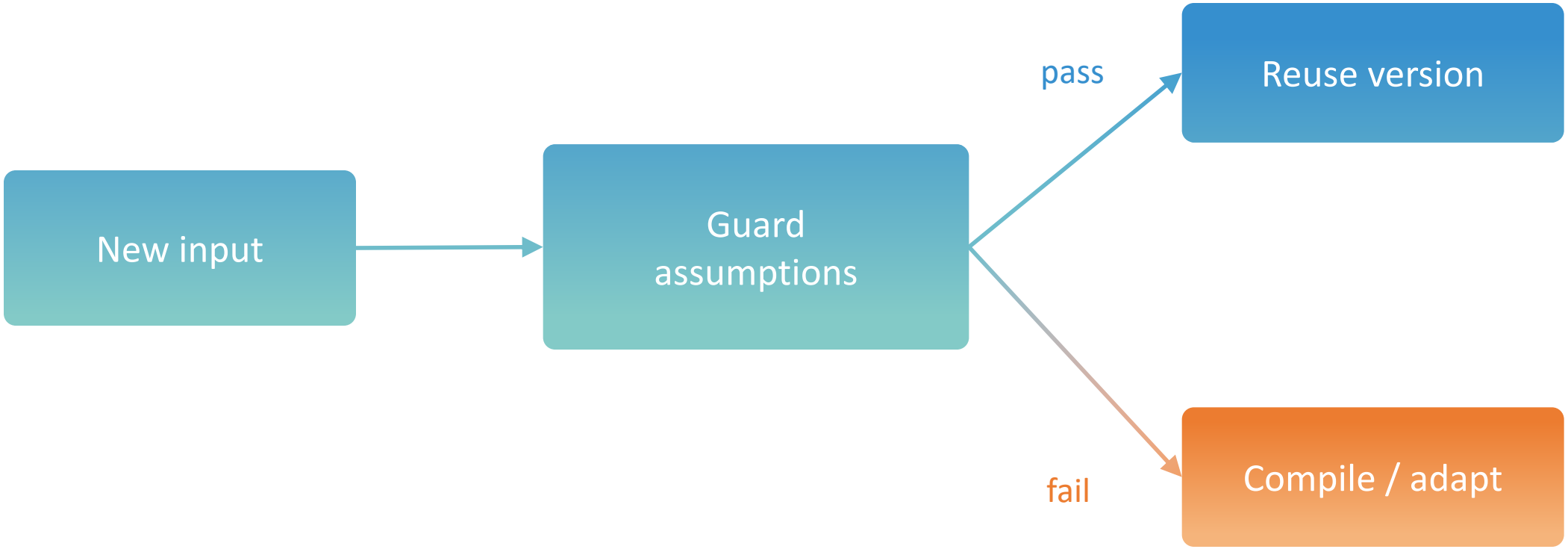
Model estimates reduce search cost; measurements decide performance.

A torch.compile Execution Path



A representative path. Backends and training paths can differ.

Guards and Shape Specialization



Reuse is conditional on the assumptions of the compiled version.

Graph Breaks Limit Optimization Scope



Fewer cross-region optimization opportunities

A graph break changes the compilation region, not necessarily correctness.

Exercise: Fusion Traffic

上海创智学院
Shanghai Innovation Institute

$y = \mathrm{ReLU}(a * x + b)$, $N = 2^{20}$, FP32

1. Three kernels versus one fused kernel
2. Count total HBM read + write bytes
3. Does the byte ratio guarantee a speedup?

Scalars resident. Intermediates materialized. Ignore cross-kernel cache hits.

Exercise: Compilation Cost

上海创智学院
Shanghai Innovation Institute

Compile: 200 ms; eager: 1 ms/call; compiled: 0.6 ms/call

1. Compare total time for R calls
2. Find the break-even invocation count
3. What changes if recompilation is needed?

Teaching times. One compilation, fixed workload, equal other costs.

Graph Compilers and Kernel Compilers

上海创智学院
Shanghai Innovation Institute

```
h = relu(X @ W1 + b1)
y = h @ W2 + b2
```

Graph / model compilation

TVM



Kernel compilation: inside one Matmul



Triton / TileLang / cuTile

Compare the compilation scope before comparing programming syntax.

TVM: Relax and TIR in One IRModule

上海创智学院 Shanghai Innovation Institute

TIR + IRModule

```
import numpy as np, tvm
from tvm import relax
from tvm.script import ir as I
from tvm.script import tir as T
from tvm.script import relax as R
@I.ir_module
class Module:
    @T.prim_func
    def add1(a: T.handle, b: T.handle):
        A = T.match_buffer(a, (8,))
        B = T.match_buffer(b, (8,))
        for i in T.serial(8):
            B[i] = A[i] + T.float32(1)
```

Relax + build + call

```
    @R.function
    def main(
        x: R.Tensor((8,), "float32")
    ):
        y = R.call_tir(
            Module.add1, (x,),
            out_sinfo=R.Tensor(
                (8,), "float32"))
        return R.nn.relu(y)
mod = relax.transform.LegalizeOps()(
    Module)
exe = tvm.compile(mod, target="llvm")
vm = relax.VirtualMachine(exe,tvm.cpu())
x = np.arange(8,dtype="float32") - 4
y = vm["main"](tvm.nd.array(x)).numpy()
np.testing.assert_allclose(
    y, np.maximum(x+1,0))
```

TVM v0.20.0 · y = ReLU(x + 1) · read left → right

Triton: Masked Memory Access

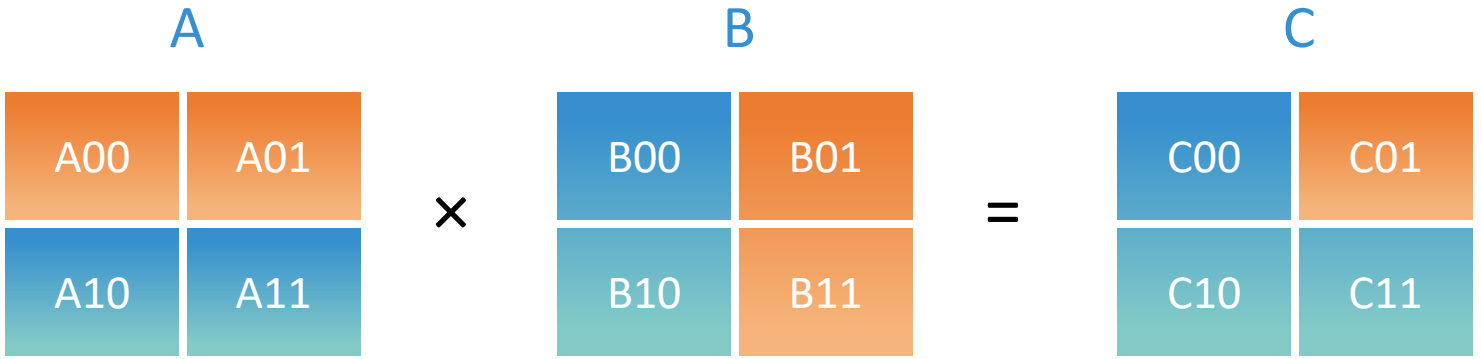
上海创智学院
Shanghai Innovation Institute

```
import torch
import triton
import triton.language as tl
@triton.jit
def add(X, Y, Z, N, B: tl.constexpr):
    i = tl.program_id(0) * B + tl.arange(0, B)
    x = tl.load(X + i, i < N, other=0)
    y = tl.load(Y + i, i < N, other=0)
    tl.store(Z + i, x + y, i < N)
x = torch.arange(10., device="cuda")
y = torch.ones_like(x)
z = torch.empty_like(x)
add[(triton.cdiv(10, 4),)](x, y, z, 10, B=4)
torch.testing.assert_close(z, x + y)
```



N = 10 · BLOCK = 4 · 3 programs, with a masked tail

Triton: Pointer Grids and Tile Operations



Program (0,1)
2 K-tiles
32 × 32 output

```
acc = tl.zeros((BM, BN), tl.float32)
for k in range(2):
  a = tl.load(ap + k * BK)
  b = tl.load(bp + k * BK * stride_bk)
  acc += tl.dot(a, b)
tl.store(cp, acc)
```

ap, bp: pointer grids
A tile: 32 × 32
B tile: 32 × 32
One program → C01

Kernel body · fixed 64 × 64 GEMM · 32 × 32 tiles

TileLang: Explicit Blocks and Parallel Loops

上海创智学院
Shanghai Innovation Institute

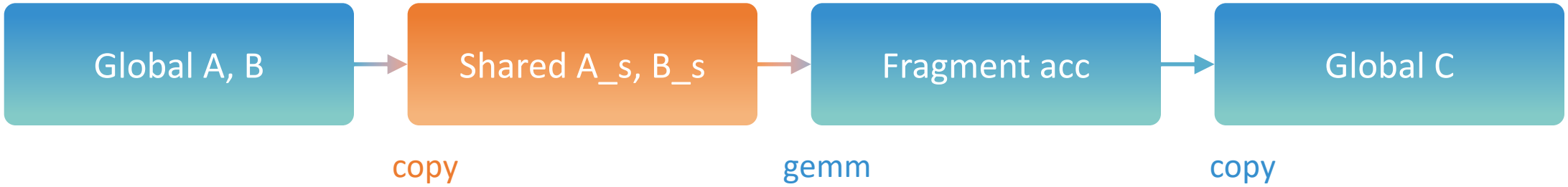
```
import torch
import tilelang
import tilelang.language as T
@tilelang.jit
def add(A, B):
    A: T.Tensor((128,), T.float32)
    B: T.Tensor((128,), T.float32)
    C = T.empty((128,), T.float32)
    with T.Kernel(1, threads=128):
        for i in T.Parallel(128):
            C[i] = A[i] + B[i]
    return C
kernel = add.compile()
a = torch.ones(128, device="cuda")
b = torch.ones_like(a)
c = kernel(a, b)
torch.testing.assert_close(c, a + b)
```

@tilelang.jit → add.compile() → kernel(a, b)

TileLang: Explicit Memory and Pipelines

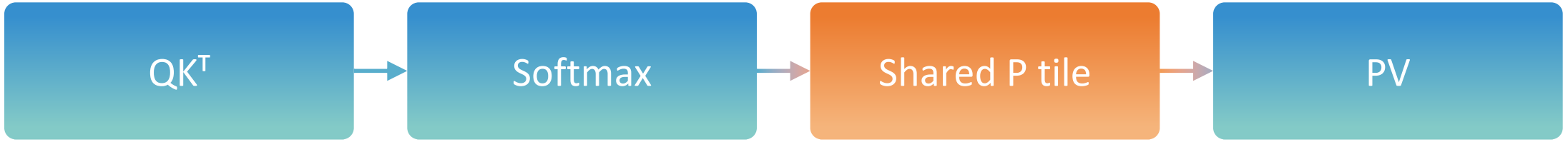
上海创智学院
Shanghai Innovation Institute

```
A_s = T.alloc_shared((32, 32), "float16")
B_s = T.alloc_shared((32, 32), "float16")
acc = T.alloc_fragment((32, 32), "float32")
T.clear(acc)
for k in T.Pipelined(2, num_stages=2):
  T.copy(A[mi*32, k*32], A_s)
  T.copy(B[k*32, ni*32], B_s)
  T.gemm(A_s, B_s, acc)
T.copy(acc, C[mi*32, ni*32])
```



Kernel body · shared buffers, fragments and a 2-stage pipeline

TileLang: Tuning MLA Layouts and Pipelines




```
S_shared = T.alloc_shared((BM, BN), dtype)
T.annotate_layout({
  S_shared: make_swizzled_layout(S_shared)
})
for k in T.Pipelined(K_tiles, num_stages=3):
  ...  # copies and GEMMs
```

Choose a shared layout
Overlap copy + compute
Control tile lifetimes

H100 MLA: reported gains over the compared Triton kernels

MLA is a workload-specific example; exposed control is not a speed guarantee.

cuTile: Tile Operations in Python

上海创智学院
Shanghai Innovation Institute

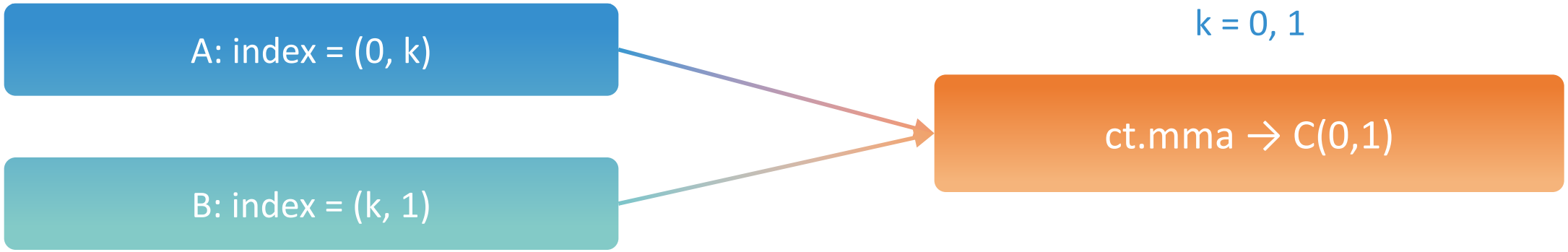
```
import torch
import cuda.tile as ct
@ct.kernel
def add(A, B, C):
    i = ct.bid(0)
    a = ct.load(A, index=(i,), shape=(128,))
    b = ct.load(B, index=(i,), shape=(128,))
    ct.store(C, index=(i,), tile=a + b)
a = torch.ones(128, device="cuda")
b = torch.ones_like(a)
c = torch.empty_like(a)
ct.launch(torch.cuda.current_stream(),
          (1, 1, 1), add, (a, b, c))
torch.testing.assert_close(c, a + b)
```

@ct.kernel → ct.launch(stream, grid, kernel, args)

cuTile: Addressing Tiles by Coordinates

上海创智学院
Shanghai Innovation Institute

```
mi, ni = ct.bid(0), ct.bid(1)
acc = ct.full((32, 32), 0, dtype=ct.float32)
for k in range(ct.num_tiles(A, axis=1, shape=(32,32))):
  a = ct.load(A, index=(mi,k), shape=(32,32))
  b = ct.load(B, index=(k,ni), shape=(32,32))
  acc = ct.mma(a, b, acc)
ct.store(C, index=(mi,ni), tile=ct.astype(acc, C.dtype))
```

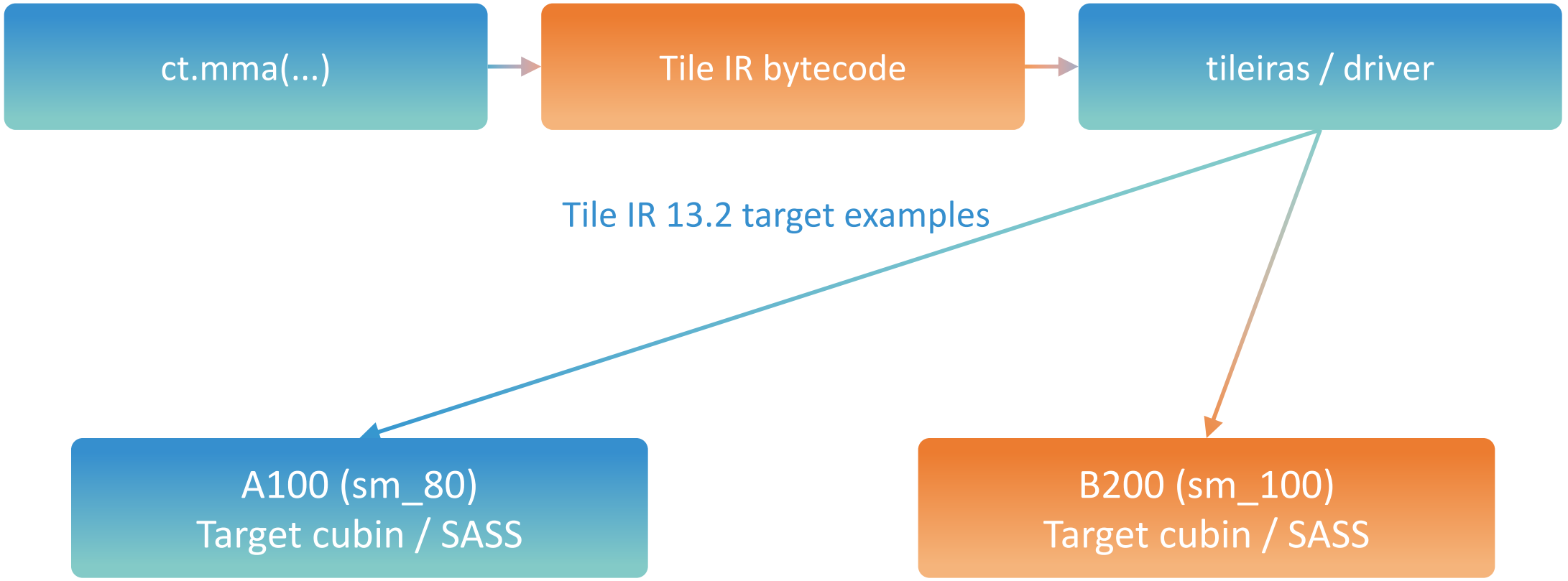


Kernel body · tile coordinates instead of pointer grids

cuTile: Tile IR Across GPU Generations

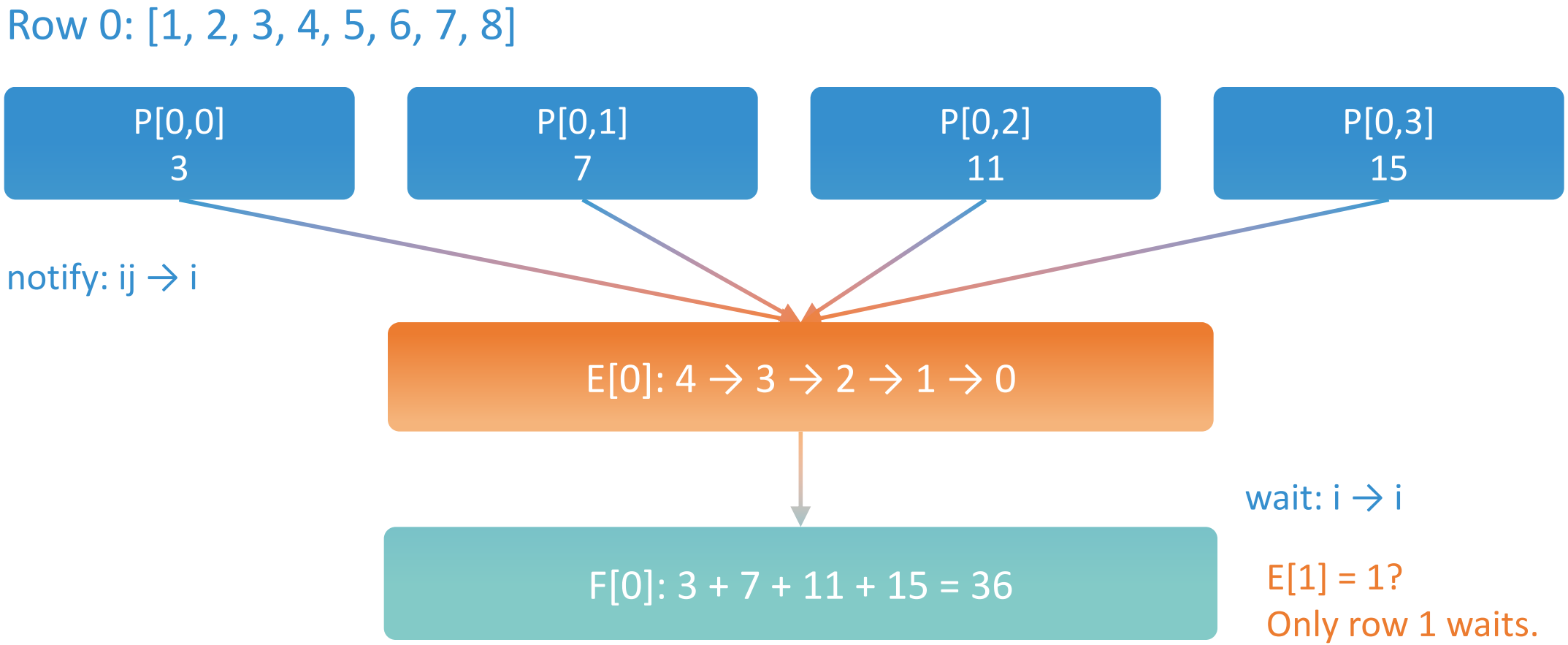
上海创智学院
Shanghai Innovation Institute

PTX also supports forward compatibility for SIMT programs.



Portability requires supported IR versions, targets and features; performance may differ.

Event Tensor: Counters Encode Readiness



An Event Tensor stores completion conditions, not numerical tensor values.

Event Tensor: Explicit Task Dependencies

上海创智学院 Shanghai Innovation Institute

Device computations

```
@device_func
def partial_sum(i, j, A, B):
    rows = slice(i*32, i*32+32)
    cols = slice(j*32, j*32+32)
    B[rows, j] = sum(
        A[rows, cols], axis=1)

@device_func
def final_sum(i, B, C):
    rows = slice(i*32, i*32+32)
    C[rows] = sum(B[rows, :], axis=1)
```

Graph dependencies

```
@graph_func
def main_graph(A, n):
    E = ETensor((n,), wait_count=4)
    B = call_device(
        partial_sum, tile_num=(n, 4),
        args=[A], in_edges={},
        out_edges={E: "ij->i"})
    C = call_device(
        final_sum, tile_num=(n,),
        args=[B], in_edges={E: "i->i"},
        out_edges={})
    return C
```

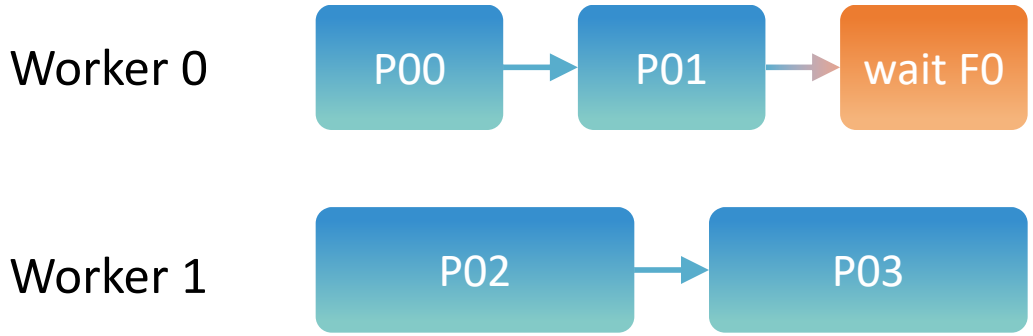
Paper pseudocode · A: (n × 32, 128) → B: (n × 32, 4) → C

Event Tensor: Static and Dynamic Scheduling

上海创智学院 Shanghai Innovation Institute

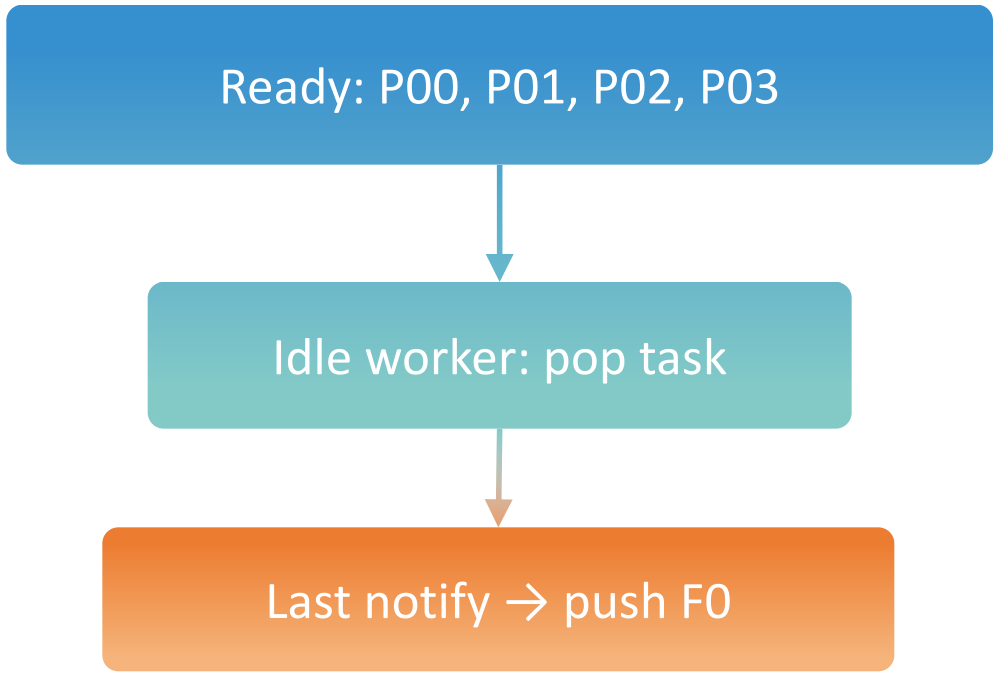
E0 counts four completions in both schedules.

Static: assigned queues



Known work distribution
Low queue overhead

Dynamic: a ready queue

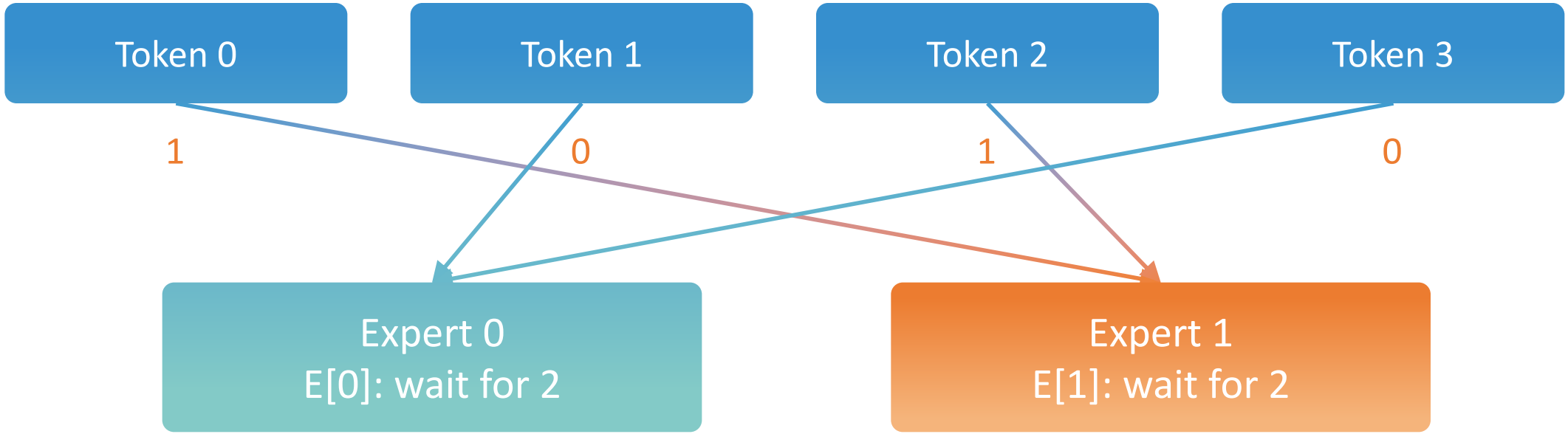


The dependency stays the same; the scheduling strategy changes.

Event Tensor: Routing Changes Dependencies

上海创智学院
Shanghai Innovation Institute

Four tokens, top-1 routing



route = [1, 1, 1, 0] on the next input → wait counts [1, 3]

The same tensor shape can have different task dependencies after routing.

PyPTO: Core Groups and Tile Operations

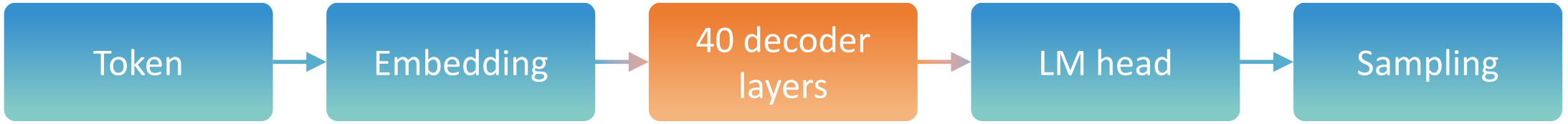
上海创智学院
Shanghai Innovation Institute

```
import torch
import pypto.language as pl
from pypto.runtime import RunConfig
@pl.jit
def add(a: pl.Tensor, b: pl.Tensor,
        c: pl.Out[pl.Tensor]):
    with pl.at(level=pl.Level.CORE_GROUP):
        x = pl.load(a, [0, 0], [128, 128])
        y = pl.load(b, [0, 0], [128, 128])
        z = pl.add(x, y)
        pl.store(z, [0, 0], c)
    return c
a = torch.ones((128, 128))
b = torch.ones_like(a)
c = torch.zeros_like(a)
add(a, b, c, config=RunConfig())
torch.testing.assert_close(c, a + b)
```

@pl.jit → add(..., config=RunConfig()) · explicit output

PyPTO: Full-Graph Fusion for Qwen Decode

上海创智学院
Shanghai Innovation Institute




```
cur = _token_embed_inline(...)
for layer_idx in pl.range(40):
  cur = _decode_layer(cur, ...)
logits = rms_lm_head_fp32(cur, ...)
next_id = _greedy_sample_inline(logits, ...)
```

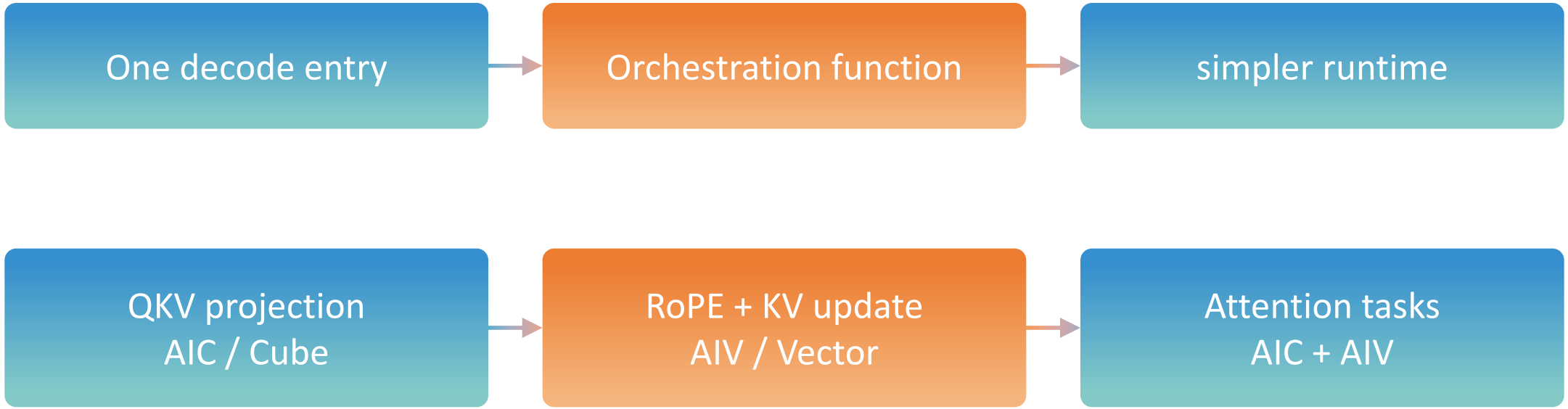
One layer: Norm → QKV / RoPE → Attention → O → MLP / Residual

pypto-lib example: Qwen3-14B, 40 layers, one fused device-side decode step.

PyPTO: One Megakernel Schedules Many Tasks

上海创智学院
Shanghai Innovation Institute

Host submits one device program



The full-graph megakernel schedules many InCore functions.

One full-graph entry can contain many dependent device tasks.

Agents Tune Under Resource Constraints

上海创智学院
Shanghai Innovation Institute

```
T.Pipelined(K_tiles,
            num_stages=3)
```

Agent proposes tile / stage / layout
Compiler reports resources
Tests check values; timer checks speed

Candidate	Shared	Fits?
2 stages	48 KiB	Yes
3 stages	72 KiB	No
½ BK, 3 stages	36 KiB	Yes

Smaller BK also means more K-loop iterations.

Teaching case: 24 KiB/stage, 64 KiB/block; no other shared allocation.

Agent Kernels Need Reference Tests

上海创智学院
Shanghai Innovation Institute

Wrong: reset inside the loop

```
for k in range(2):
  acc = 0
  acc += partial[k]
return acc
```

partial = [2, 3]
Output = 3

Correct: reset once

```
acc = 0
for k in range(2):
  acc += partial[k]
return acc
```

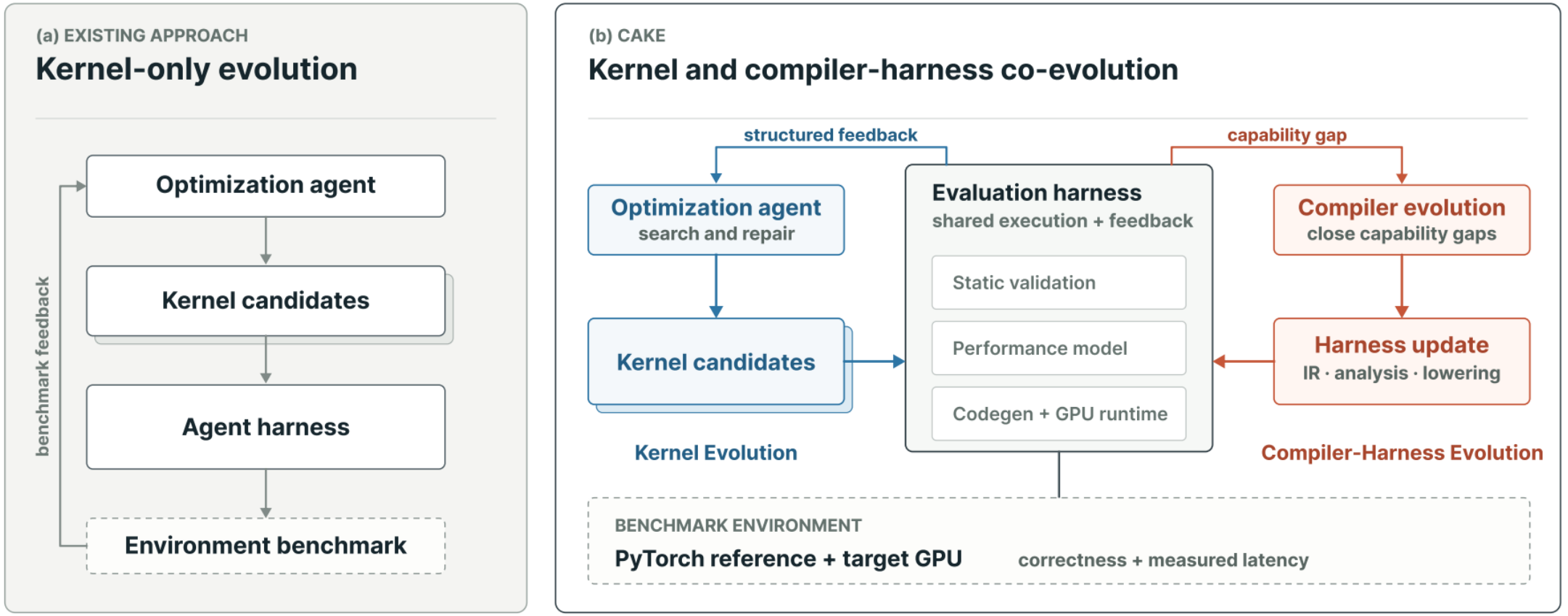
partial = [2, 3]
Output = 5

Both versions can compile; the reference test distinguishes 3 from 5.

CAKE: Evolving Kernels and Compilers

上海创智学院
Shanghai Innovation Institute

Original paper, Figure 1



Inner loop: improve a kernel. Outer loop: improve the compiler environment.

CAKE: Kernel Patterns Shape the IR




```
# A declared TMEM accumulator
acc = lm.tmem(cols=0, width=128,
              shape=(128,128), dtype=lm.f32)
```



A new primitive needs semantics, legality checks and lowering together.

CAKE: Explicit Resources and Synchronization

上海创智学院 Shanghai Innovation Institute

Resources and roles

```
@cake.schedule()
def schedule(lm, Q):
    pool = lm.smem(98304)
    smem_q = pool.view(
        offset=0, shape=(128,128),
        dtype=lm.bf16, stage=3)
    tmem_acc = lm.tmem(
        cols=0, width=128,
        shape=(128,128), dtype=lm.f32)
    load = lm.role(warps=[0])
    mma = lm.role(warps=[1])
    pipe = lm.pipeline(stages=3)
    q_full = lm.barrier(
        count=3, prod=[load], cons=[mma],
        init_count=1, pipeline=pipe)
```

Producer and consumer

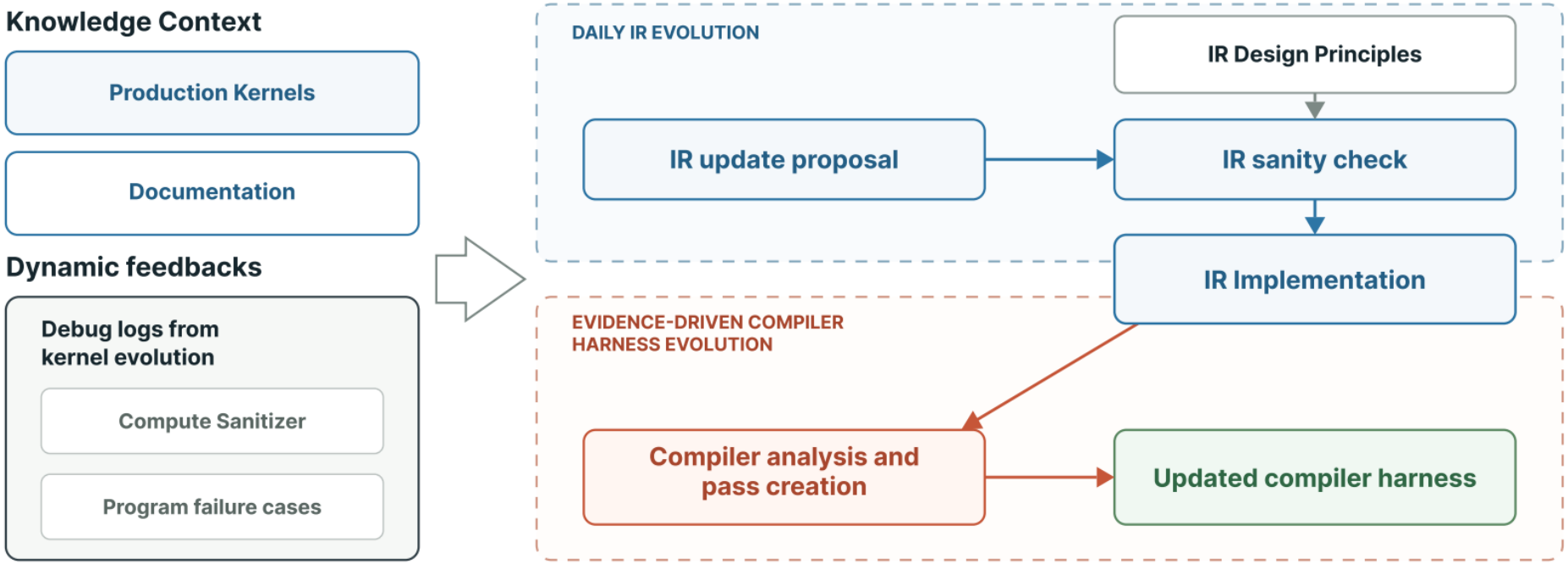
```
with load:
    for stage in lm.range(0, 3):
        smem_q.tma_load(
            Q, coords=(0,0,stage),
            stage=stage,
            barrier=q_full)
with mma:
    for stage in lm.range(0, 3):
        lm.wait(q_full, stage=stage)
        lm.fence_proxy()
        lm.mma(
            tmem_acc, smem_q[stage],
            smem_q[stage],
            init=(stage == 0))
```

Paper schedule fragment · 3 × 32 KiB buffers · not full FMHA

CAKE: Failures Become Compiler Checks

上海创智学院 Shanghai Innovation Institute

Original paper, Figure 4



Buffer reused too early → Lifetime check → Valid + invalid tests

A repeated buffer-lifetime failure can become a reusable static check.

CAKE: KMeans as GEMM and Reduction

上海创智学院
Shanghai Innovation Institute

Points

x0	1	0
x1	3	0

Centroids

c0	0	0
c1	4	0

Squared distances

1	9
9	1

argmin = [0, 1]

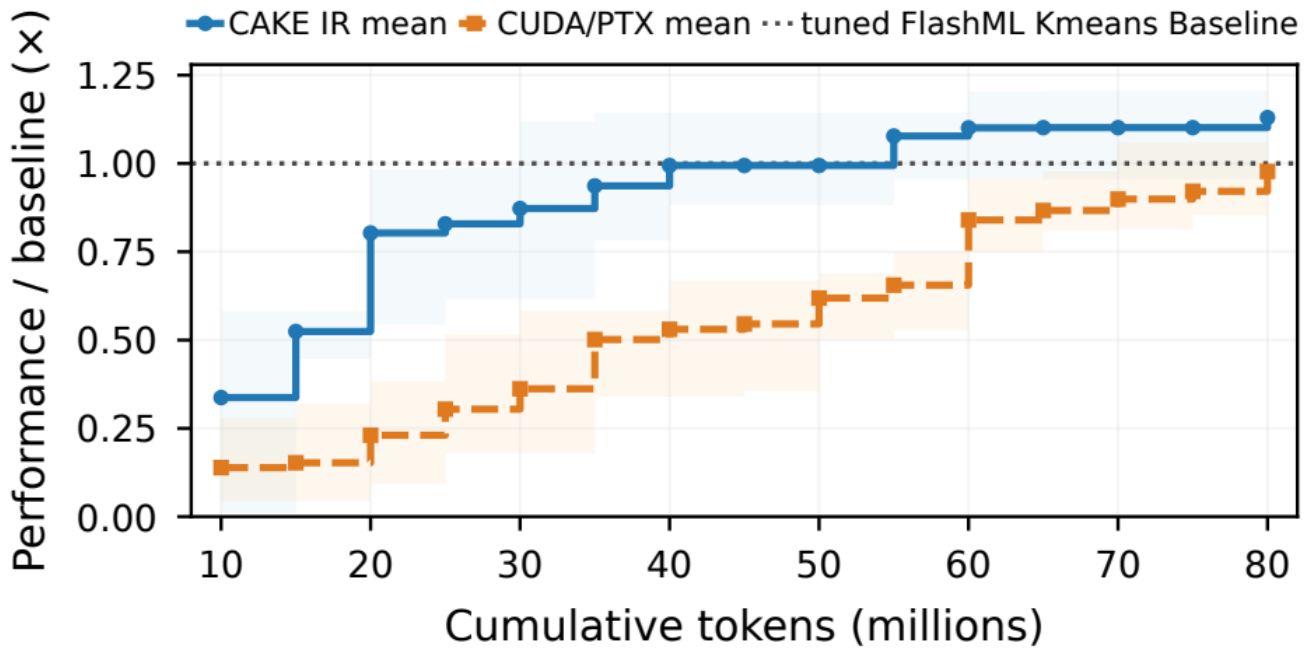


Paper workload: B = 32, N = 65,536, K = 1,024, D = 128; BF16 → FP32.

CAKE: IR Improves KMeans Search

上海创智学院
Shanghai Innovation Institute

Figure 5: mean curves, min–max bands



Interface	Median best
CAKE IR	1.144×
CUDA / PTX	0.928×

80M-token endpoint
Table 2: median of 3 runs

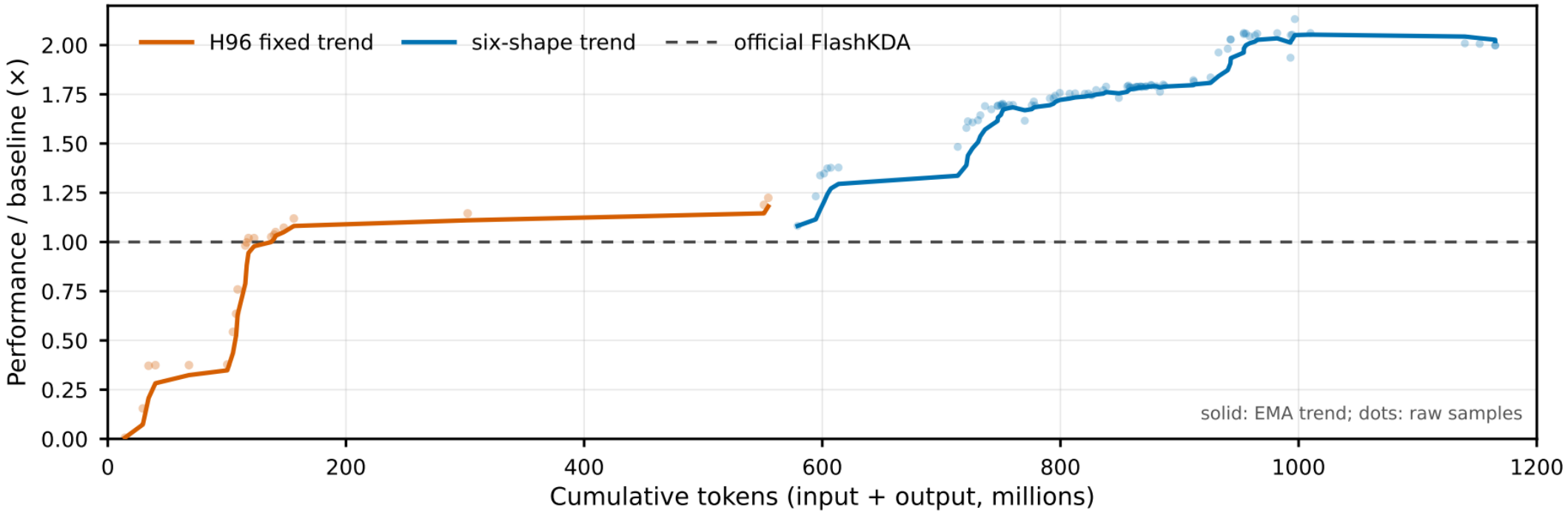
Baseline: tuned FlashML KMeans, 0.938 ms

B200; 3 runs/arm; 80M tokens/run; low-level target implementations hidden.

CAKE: Optimizing KDA Across Shapes

上海创智学院
Shanghai Innovation Institute

Kimi Delta Attention: state survives across chunks



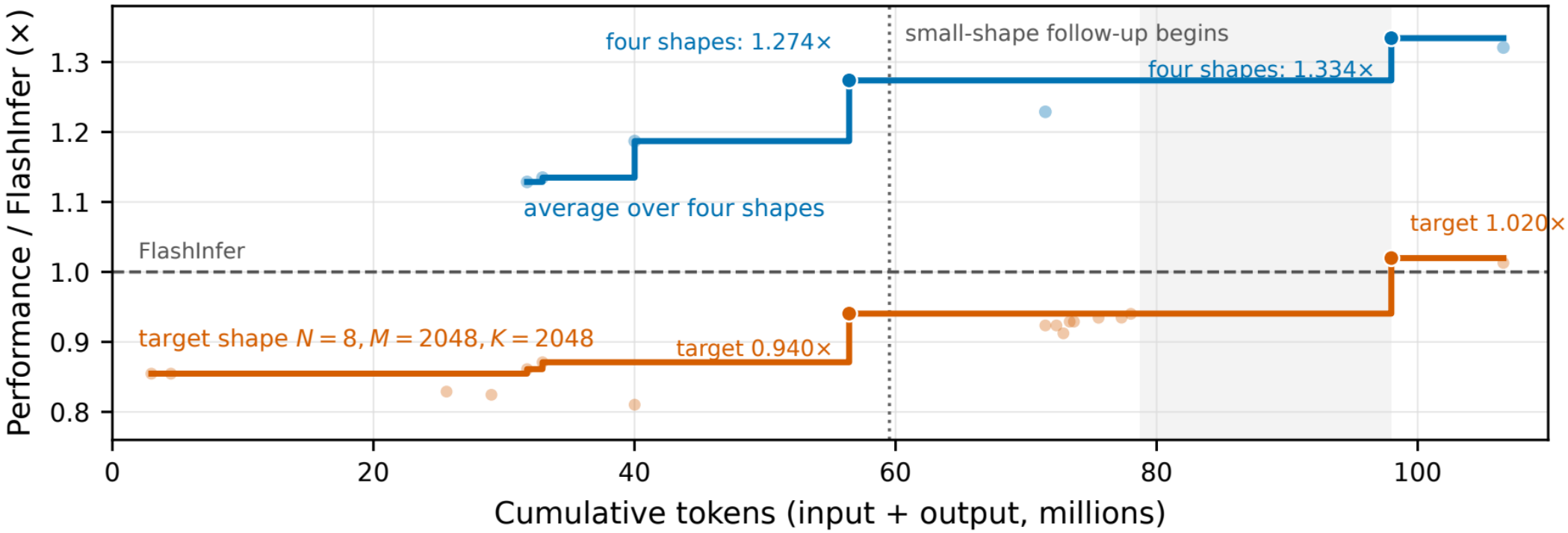
Bring-up on one shape → optimize six shapes → validate in SGLang

Orange: one shape. Blue: six-shape geometric mean. The objectives differ.

CAKE: Mean Speedups Can Hide Regressions

上海创智学院
Shanghai Innovation Institute

TinyGEMM: a strong mean can hide a slow small-N shape



N = 8, M = 2,048, K = 2,048: target the regression, then recheck all shapes.

Four-shape mean: 1.274× → 1.334×; the slow target: 0.940× → 1.020×.

Review: Representation and Optimization

上海创智学院
Shanghai Innovation Institute

Represent	Expose the decisions that matter
Transform	Preserve the required semantics
Evaluate	Measure benefit and total cost
Reuse	Retain useful implementations and rules

Explain an optimization through its conditions, mechanism and evidence.

Acknowledgement

The development of this course, including its structure, content, and accompanying presentation slides, has been significantly influenced and inspired by the excellent work of instructors and institutions who have shared their materials openly. We wish to extend our sincere acknowledgement and gratitude to the following courses, which served as invaluable references and a source of pedagogical inspiration:

- Machine Learning Systems[15-442/15-642], by **Tianqi Chen** and **Zhihao Jia** at **CMU**.
- Advanced Topics in Machine Learning (Systems)[CS6216], by **Yao Lu** at **NUS**

While these materials provided a foundational blueprint and a wealth of insightful examples, all content herein has been adapted, modified, and curated to meet the specific learning objectives of our curriculum. Any errors, omissions, or shortcomings found in these course materials are entirely our own responsibility. We are profoundly grateful for the contributions of the educators listed above, whose dedication to teaching and knowledge-sharing has made the creation of this course possible.

System for Artificial Intelligence

Thanks

Siyuan Feng
Shanghai Innovation Institute